



CSE276: ARTIFICIAL INTELLIGENCE FOUNDATIONS

The Complete Self-Learning Guide for LPU Students

By Your Friendly AI Senior



PREFACE: How to Use This Book

Hey there! Welcome to your AI journey.

If you are reading this, chances are you are an LPU student preparing for CSE276 — Artificial Intelligence Foundations. Maybe you are worried about the 25-mark Continuous Assessment (CA). Maybe you have tried reading heavy textbooks and felt lost. Maybe you just want someone to explain things like a real human being, not a robot.

This book is exactly for you.

I have written this book like a senior sitting next to you in the hostel or library, explaining every concept patiently, using simple words, everyday examples, and a lot of analogies. No heavy jargon. No confusing paragraphs. Just clear, spoken-style English that you can even read aloud.

How This Book is Organized

Every unit in this book matches your official LPU syllabus exactly. Nothing is skipped. Nothing is rearranged.

For every topic, you will find:

- **One-line definition** — so you know what we are talking about
- **Why it matters** — so you know why you are learning it
- **An easy analogy** — so the concept sticks in your brain
- **Step-by-step explanation** — so you understand the “how”
- **Practical examples** — so you see it in action
- **Common mistakes** — so you do not lose marks foolishly
- **Quick summary** — for last-minute revision

At the end of every unit:

- **Key Points to Remember** (5–8 bullets)
- **Self-Check Questions** (short answer)
- **Scenario-Based MCQs** (8–12 questions, exactly like LPU CA style)

At the very end:

- **Full Practice Test** (25 scenario-based MCQs)
- **Final One-Page Revision Sheet**

How to Score Well in the 25-Mark CA

LPU CA questions are **scenario-based MCQs**. This means they will NOT ask you “Define AI.” Instead, they will give you a situation like:

“A hospital wants to build a system that predicts diseases from symptoms. Which type of AI agent is most suitable?”

To answer these, you need **deep understanding**, not rote memorization.

My advice:

1. Read one unit at a time. Do not rush.
2. Read the analogy first. It builds your mental model.
3. Try to explain the concept to yourself in your own words.
4. Solve ALL scenario-based MCQs at the end of each unit.
5. Read the explanations carefully — even for the wrong options. That is where learning happens.
6. Before the exam, read only the “Key Points to Remember” and the “Final Revision Sheet.”

Let’s begin!

TABLE OF CONTENTS

Unit	Topic
Unit I	Foundations of Artificial Intelligence
Unit II	AI Problem Solving
Unit III	Problem Solving Using Search Techniques
Unit IV	Knowledge Representation and Reasoning
Unit V	Intelligent Agents
Unit VI	Evolution of Modern AI
End	Full Practice Test + Final Revision Sheet

UNIT I: FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

1.1 Introduction to AI: History and Evolution

Simple Definition

Artificial Intelligence (AI) is the ability of a machine or computer to think, learn, and make decisions like a human being.

Why It Matters

AI is everywhere today. When you use Google Maps, when Netflix suggests a movie, when your phone unlocks with your face — that is AI. Understanding its history helps you see how we reached today's powerful AI tools.

Easy Analogy

Think of AI like a student. In the beginning, the student knows nothing (like a blank slate). Then teachers give books (data), the student studies (learns patterns), and finally the student can answer exam questions (make predictions or decisions). AI is that student, but it is a computer.

Detailed Explanation: The Story of AI (Step by Step)

Step 1: The Dream (1950s) In 1950, a British mathematician named **Alan Turing** asked a simple but powerful question: *“Can machines think?”* He created the **Turing Test** — if a machine can chat with a human and the human cannot tell it is a machine, then the machine is intelligent.

Imagine you are texting someone. If you cannot tell whether the person on the other side is a human or a robot, and it is actually a robot, then that robot has passed the Turing Test.

Step 2: The Birth (1956) At a conference at **Dartmouth College** in 1956, the term “Artificial Intelligence” was officially born. Scientists like John McCarthy, Marvin Minsky, and others said: *“We believe we can make machines use language, form concepts, and solve problems.”*

Step 3: Early Enthusiasm (1960s–1970s) Scientists built programs that could solve algebra problems, play chess, and understand simple English. But these programs were **rule-based** — meaning humans had to write thousands of “if-then” rules manually.

Think of it like teaching a child by writing down every possible situation: “If you see a red light, then stop.” But what about orange lights? Flashing lights? The list became too long.

Step 4: The First AI Winter (1970s–1980s) People realized that writing all rules by hand was impossible for complex real-world problems. Funding dried up. This period is called the **AI Winter** — like winter when nothing grows.

Step 5: Expert Systems (1980s) Then came **expert systems**. These were programs that acted like human experts (doctors, engineers) using knowledge bases. They worked well in narrow areas but failed outside their specific domain.

Step 6: Machine Learning Revolution (1990s–2000s) Scientists changed their approach. Instead of writing all rules, they said: *“Let the machine learn from data.”* This gave birth to **Machine Learning (ML)**. Now the machine could look at thousands of examples and find patterns on its own.

Step 7: Deep Learning Boom (2010s–Present) With more data and powerful computers (GPUs), scientists built **Deep Learning** — AI that uses artificial neural networks with many layers. This led to breakthroughs in image recognition, language translation, and self-driving cars.

Step 8: Generative AI Era (2020s–Now) Today we have **ChatGPT, Gemini, Claude** — AI that can write essays, generate images, and even write code. This is called **Generative AI**.

Practical Example

Example 1: In 1997, IBM's Deep Blue defeated world chess champion Garry Kasparov. This was a milestone because it showed AI could beat humans in complex thinking tasks.

Example 2: In 2016, Google’s AlphaGo defeated the world champion of Go — a game far more complex than chess. AlphaGo used deep learning and played millions of games against itself to learn.

Common Mistakes Students Make

- **✗** Thinking AI is only robots. AI is software too — like Siri, Google Search, and spam filters.
- **✗** Confusing AI with Machine Learning. ML is a *subset* of AI. All ML is AI, but not all AI is ML.
- **✗** Believing AI can think like humans. AI only mimics intelligence. It does not have consciousness or feelings.

Quick Summary

- AI = machines acting smart
 - History: Turing Test → Dartmouth → Expert Systems → Machine Learning → Deep Learning → Generative AI
 - AI winters happened when expectations were too high and technology was too weak
 - Today AI is everywhere: phones, cars, hospitals, banks
-

1.2 Characteristics and Types of AI

Simple Definition

AI systems have certain characteristics that make them “intelligent,” and they can be classified into different types based on what they can do.

Why It Matters

Knowing the types of AI helps you understand what current AI can and cannot do. It also helps in answering scenario questions like: “Which type of AI is suitable for this task?”

Easy Analogy

Think of AI types like grades in school:

- **Narrow AI** = A student who is brilliant in one subject (like math) but knows nothing else.
- **General AI** = A student who is good at ALL subjects — like a true all-rounder.
- **Super AI** = A student who is better than every teacher in every subject. (This does not exist yet.)

Detailed Explanation

Characteristics of AI An AI system is considered intelligent if it shows these characteristics:

1. **Learning:** The ability to improve from experience. Like you getting better at cricket after every match.
2. **Reasoning:** The ability to draw conclusions from facts. Like solving a puzzle.
3. **Problem-Solving:** Finding solutions to difficult situations. Like finding the shortest route home.
4. **Perception:** Understanding the world through sensors (cameras, microphones). Like self-driving cars “seeing” the road.

5. **Language Understanding:** Understanding and generating human language. Like ChatGPT reading your question.

Types of AI (Based on Capability) Type 1: Artificial Narrow Intelligence (ANI)

- Also called **Weak AI**
- Can do ONE specific task very well
- Examples: Siri, Alexa, Google Translate, spam filters, chess-playing programs
- **All AI we have today is Narrow AI**

Type 2: Artificial General Intelligence (AGI)

- Also called **Strong AI**
- Can understand, learn, and apply intelligence across ANY task — just like a human
- Does NOT exist yet
- If built, it could switch from playing chess to writing poetry to diagnosing diseases without being reprogrammed

Type 3: Artificial Super Intelligence (ASI)

- AI that surpasses human intelligence in EVERY field
- Smarter than the smartest human in science, art, strategy, everything
- Purely theoretical right now
- Some scientists worry about safety if ASI is ever created

Types of AI (Based on Functionality) Reactive Machines

- The simplest type
- React to current input only
- Have NO memory of past events
- Example: IBM's Deep Blue. It looked at the current chess board and made the best move. It did not remember previous games.

Limited Memory

- Can use past data to make better decisions
- Have short-term memory
- Example: Self-driving cars. They remember the speed and position of nearby cars from the last few seconds to drive safely.
- **Most modern AI falls here.**

Theory of Mind

- AI that understands human emotions, beliefs, and thoughts
- Can interact socially like a human
- Does NOT exist yet

- Example in movies: JARVIS from Iron Man (partially), or Samantha from the movie “Her”

Self-Aware AI

- AI that has consciousness and self-awareness
- It knows it is a machine and has its own desires
- Purely science fiction right now
- Example: The Terminator (but hopefully not that evil!)

Practical Example

Example 1: A chatbot that answers bank queries is **Narrow AI** and **Limited Memory** type. It is narrow because it only handles banking. It has limited memory because it remembers your last few messages in the conversation.

Example 2: A robot that vacuums your room (like Roomba) is **Narrow AI** and **Reactive Machine** (older models) or **Limited Memory** (newer models that remember room layout).

Common Mistakes Students Make

- ❌ Thinking Siri or Alexa is General AI. No — they are Narrow AI. Ask Siri to drive your car, and it will fail.
- ❌ Confusing “Theory of Mind” with chatbots understanding emotions. Chatbots only *pretend* to understand. They do not actually feel or believe anything.
- ❌ Forgetting that ALL current AI is Narrow AI. This is a favorite exam trap.

Quick Summary

- AI characteristics: Learning, Reasoning, Problem-Solving, Perception, Language Understanding
- By capability: ANI (exists), AGI (future), ASI (far future)
- By functionality: Reactive, Limited Memory, Theory of Mind, Self-Aware
- Every AI you use today = Narrow + Limited Memory

1.3 AI vs Machine Learning vs Deep Learning vs Data Science

Simple Definition

These four terms are related but NOT the same. Think of them as nested circles — AI is the biggest, and the others live inside it.

Why It Matters

In job interviews and exams, you MUST know the difference. A scenario might ask: “A company wants to predict sales from past data. Should they use AI, ML, or DL?”

Easy Analogy

Imagine a **university**:

- **AI** = The entire university (all departments)

- **Machine Learning** = The Science Department (a big part of the university)
- **Deep Learning** = The Physics Lab inside the Science Department (a specialized branch)
- **Data Science** = A student who takes courses from ALL departments — science, arts, math — to solve real-world problems

Detailed Explanation

Artificial Intelligence (AI)

- The **BROADEST** term
- Any technique that makes machines act smart
- Includes rule-based systems, expert systems, machine learning, robotics, natural language processing
- Example: A chess program that uses hand-written rules is AI but NOT Machine Learning

Machine Learning (ML)

- A **subset** of AI
- Instead of writing rules, we feed data to the machine and let it find patterns
- The machine “learns” from examples
- Three types: Supervised, Unsupervised, Reinforcement Learning
- Example: Showing a machine 10,000 pictures of cats and dogs so it learns to tell them apart

Deep Learning (DL)

- A **subset** of Machine Learning
- Uses artificial neural networks with MANY layers (hence “deep”)
- Needs LOTS of data and powerful computers
- Excellent for images, speech, and language
- Example: Face recognition on your phone, voice assistants understanding your accent

Data Science

- NOT exactly a subset of AI — it is a **broader field**
- Uses statistics, math, programming, AND AI/ML to extract insights from data
- A data scientist might use simple charts (not AI at all) or complex deep learning models
- Focuses on asking the right questions, cleaning data, and making business decisions
- Example: A data scientist analyzing which products sell best during Diwali using Excel, SQL, and maybe some ML

Visual Description (Imagine This)

Draw four circles:

- The biggest circle is **AI**
- Inside AI, draw a smaller circle called **ML**
- Inside ML, draw an even smaller circle called **DL**
- Now draw a circle called **Data Science** that **OVERLAPS** with AI but also extends outside it (because data science uses non-AI tools too)

Practical Example

Scenario: A hospital has patient records and wants to predict which patients are at risk of diabetes.

- **Data Science approach:** Clean the data, find patterns using statistics, create charts for doctors.
- **ML approach:** Train a model on past patient data to predict diabetes risk.
- **DL approach:** Use a complex neural network (maybe overkill for this simple task, but useful if analyzing X-ray images).
- **AI approach:** The overall system that includes the ML model, a chatbot for patients, and a dashboard for doctors.

Comparison Table

Feature	AI	ML	DL	Data Science
Definition	Machines acting smart	Machines learning from data	Deep neural networks learning	Extracting insights from data
Scope	Broadest	Subset of AI	Subset of ML	Overlaps with AI
Data needed	Can work with little data	Needs moderate data	Needs LOTS of data	Any amount
Human involvement	High (rules) or Low	Medium (feature engineering)	Low (auto feature extraction)	High (asking questions)
Example	Chess program with rules	Spam filter	Face recognition	Sales trend analysis

Common Mistakes Students Make

- ❌ Using AI and ML interchangeably. They are NOT the same. ML is one way to achieve AI.
- ❌ Thinking Deep Learning is always better. For small data, simple ML works better.
- ❌ Believing Data Science = AI. Data science is broader and includes non-AI work.

Quick Summary

- AI = Big umbrella (all smart machines)
- ML = Learning from data (inside AI)
- DL = Deep neural networks (inside ML)
- Data Science = Using data to solve problems (overlaps with AI but is broader)

1.4 Intelligent Systems and AI Lifecycle

Simple Definition

An **intelligent system** is a complete setup where AI technology works with hardware and software to solve a real problem. The **AI lifecycle** is the step-by-step process of building such a system.

Why It Matters

Companies do not just “build AI.” They follow a process. Understanding this lifecycle helps you answer scenario questions about how AI projects are managed.

Easy Analogy

Think of building an intelligent system like **opening a dhaba (roadside eatery)**:

1. First, you decide what food to serve (problem definition)
2. Then you collect ingredients (data collection)
3. You clean and chop vegetables (data preprocessing)
4. You cook the food (model building)
5. You taste it and adjust salt (model evaluation)
6. You serve customers (deployment)
7. You ask for feedback and improve (monitoring)

Detailed Explanation

What is an Intelligent System? An intelligent system has three parts:

1. **Sensors/Input:** How the system gets information (camera, microphone, keyboard, database)
2. **Processing/Brain:** The AI algorithm that thinks and decides (ML model, rules engine, neural network)
3. **Actuators/Output:** How the system acts on its decision (screen display, robot arm, alert message)

Example: A smart traffic signal

- **Sensors:** Cameras that count cars
- **Brain:** AI that decides how long each signal should stay green
- **Actuators:** The signal lights that change based on the decision

The AI Lifecycle (Step by Step) Step 1: Problem Definition

- What problem are we solving?
- Who will use this AI?
- What does success look like?
- Example: “We want to reduce customer complaints by 30% using an AI chatbot.”

Step 2: Data Collection

- Gather relevant data from databases, sensors, or the internet
- The quality of AI depends heavily on data quality
- Example: Collecting 50,000 past customer service chats

Step 3: Data Preprocessing

- Clean the data (remove duplicates, fix errors)

- Handle missing values
- Convert data into a format the AI can understand
- Example: Removing chats with bad spelling, converting text to numbers

Step 4: Model Selection and Training

- Choose an algorithm (decision tree, neural network, etc.)
- Train the model on the prepared data
- The model learns patterns during training
- Example: Training a neural network to classify customer queries

Step 5: Model Evaluation

- Test the model on NEW data it has never seen
- Check accuracy, precision, recall
- If performance is poor, go back to Step 3 or 4
- Example: Testing if the chatbot correctly answers 90% of questions

Step 6: Deployment

- Put the model into real-world use
- Connect it to the website, app, or hardware
- Example: The chatbot goes live on the company website

Step 7: Monitoring and Maintenance

- Watch how the AI performs in real life
- Retrain with new data periodically
- Fix bugs and improve
- Example: Noticing the chatbot confuses “refund” with “exchange” and fixing it

Practical Example

Example: Building an AI that detects crop diseases from leaf photos.

1. **Problem:** Farmers cannot identify diseases early, leading to crop loss.
2. **Data:** 10,000 photos of healthy and diseased leaves.
3. **Preprocessing:** Resize all photos to the same size, remove blurry ones.
4. **Training:** Train a Convolutional Neural Network (CNN — a type of deep learning).
5. **Evaluation:** Test on 2,000 new leaf photos. Achieve 95% accuracy.
6. **Deployment:** Build a mobile app where farmers can upload leaf photos.
7. **Monitoring:** Farmers report that the app confuses two similar diseases. Retrain with more examples.

Common Mistakes Students Make

- **✗** Thinking deployment is the last step. Monitoring is equally important.
- **✗** Ignoring data quality. Bad data = Bad AI, no matter how good the algorithm.

- ❌ Skipping problem definition. Many AI projects fail because nobody asked “What problem are we actually solving?”

Quick Summary

- Intelligent System = Sensors + Brain (AI) + Actuators
 - AI Lifecycle = Problem → Data → Preprocessing → Training → Evaluation → Deployment → Monitoring
 - Data quality is more important than algorithm choice
 - AI is never “done” — it needs continuous improvement
-

1.5 Applications of AI

Simple Definition

AI is used in almost every industry today to make work faster, cheaper, and smarter.

Why It Matters

Exam questions often give a scenario from a specific industry and ask which AI technique fits best. You need to know real-world applications.

Easy Analogy

AI is like **electricity**. Just as electricity powers fans, lights, and factories, AI powers healthcare, finance, education, and more. It is a general-purpose technology.

Detailed Explanation

1. Healthcare

- **Disease Detection:** AI analyzes X-rays, MRIs, and CT scans to find tumors, fractures, and infections faster than doctors.
- **Drug Discovery:** AI predicts which chemical compounds might become effective medicines, saving years of lab work.
- **Personalized Treatment:** AI suggests treatments based on a patient’s genes and medical history.
- **Example:** Google’s AI can detect diabetic eye disease from retina scans with 90%+ accuracy.

2. Agriculture

- **Crop Monitoring:** Drones with AI cameras check crop health from the sky.
- **Yield Prediction:** AI predicts how much crop will grow based on weather and soil data.
- **Pest Detection:** AI identifies pests and suggests targeted pesticide use.
- **Example:** Farmers in India use apps like CropIn to get AI-based advisory on when to sow and irrigate.

3. Finance

- **Fraud Detection:** AI spots unusual transactions (like your card being used in two countries at once) and blocks them.
- **Algorithmic Trading:** AI buys and sells stocks in milliseconds based on market patterns.

- **Credit Scoring:** AI assesses loan risk by analyzing spending habits, not just salary slips.
- **Example:** Paytm and PhonePe use AI to detect fraudulent transactions in real-time.

4. Education

- **Personalized Learning:** AI tutors adapt to each student's speed. If you are weak in calculus, the AI gives more calculus problems.
- **Automated Grading:** AI grades objective and even some subjective answers.
- **Plagiarism Detection:** Tools like Turnitin use AI to find copied content.
- **Example:** BYJU'S and Khan Academy use AI to recommend lessons based on student performance.

5. Manufacturing

- **Predictive Maintenance:** AI predicts when a machine will break down BEFORE it breaks, saving repair costs.
- **Quality Control:** AI cameras spot defective products on the assembly line.
- **Supply Chain Optimization:** AI predicts demand and manages inventory.
- **Example:** Tesla's factories use AI robots to assemble cars with extreme precision.

6. Robotics

- **Industrial Robots:** AI-powered arms weld, paint, and assemble products.
- **Service Robots:** Robots deliver food in hotels, clean floors, and guide visitors.
- **Surgical Robots:** AI assists surgeons in precise operations.
- **Example:** Da Vinci surgical robot helps doctors perform complex surgeries with tiny incisions.

7. Smart Cities

- **Traffic Management:** AI adjusts traffic signals based on real-time congestion.
- **Waste Management:** AI optimizes garbage collection routes.
- **Energy Management:** AI controls street lights and building temperatures to save electricity.
- **Public Safety:** AI cameras detect crimes or accidents and alert police.
- **Example:** Singapore uses AI to manage traffic flow and reduce congestion by 25%.

Practical Example

Scenario: A city wants to reduce electricity bills in government buildings.

- **AI Solution:** Install smart meters and use AI to learn when rooms are empty. The AI automatically turns off AC and lights in empty rooms.
- **Benefit:** 20–30% reduction in electricity bills.

Common Mistakes Students Make

- **✗** Thinking AI replaces humans completely. In most cases, AI *assists* humans (doctors, teachers, farmers).
- **✗** Not connecting the application to the right AI type. For image-based tasks, think Deep Learning. For rule-based tasks, think Expert Systems.
- **✗** Forgetting that AI in agriculture and healthcare needs LOTS of labeled data, which is often hard to get.

Quick Summary

- Healthcare: Diagnosis, drug discovery, personalized medicine
 - Agriculture: Crop monitoring, yield prediction, pest detection
 - Finance: Fraud detection, trading, credit scoring
 - Education: Personalized learning, grading, plagiarism detection
 - Manufacturing: Predictive maintenance, quality control
 - Robotics: Industrial, service, surgical robots
 - Smart Cities: Traffic, waste, energy, safety management
-

UNIT I: KEY POINTS TO REMEMBER

- AI = machines that can think, learn, and decide like humans
 - Alan Turing proposed the Turing Test in 1950; the term “AI” was coined at Dartmouth in 1956
 - AI has gone through winters (periods of low funding) and summers (periods of high progress)
 - All AI today is **Narrow AI** (ANI); AGI and ASI do not exist yet
 - By functionality: Reactive → Limited Memory → Theory of Mind → Self-Aware
 - AI ML DL; Data Science overlaps with AI but is broader
 - Intelligent System = Sensors + Processing + Actuators
 - AI Lifecycle: Problem → Data → Preprocessing → Training → Evaluation → Deployment → Monitoring
 - AI applications span healthcare, agriculture, finance, education, manufacturing, robotics, and smart cities
 - AI assists humans; it rarely replaces them completely
-

UNIT I: SELF-CHECK QUESTIONS

1. What is the Turing Test, and why is it important?
 2. Name three characteristics of an intelligent system.
 3. What is the difference between Narrow AI and General AI?
 4. Why is data preprocessing important in the AI lifecycle?
 5. Give two real-world examples of AI in agriculture.
 6. How is Machine Learning different from traditional rule-based AI?
 7. What type of AI is a self-driving car, and why?
 8. List the steps of the AI lifecycle in order.
-

UNIT I: SCENARIO-BASED MCQs

Question 1

A bank wants to build a system that automatically flags suspicious credit card transactions. The system should learn from past fraud cases and improve over time. Which combination best describes this system?

- A) It is a reactive machine and uses rule-based AI only.

- B) It is a limited memory AI system that uses Machine Learning.
- C) It is an Artificial General Intelligence system using Deep Learning.
- D) It is a self-aware AI using symbolic reasoning.

Correct Answer: B

Explanation: The system learns from past data (fraud cases) and improves — this is Machine Learning. It remembers recent transaction patterns to make decisions — this is Limited Memory AI. Option A is wrong because rule-based systems cannot learn from new data. Option C is wrong because AGI does not exist yet, and this is a narrow task (fraud detection only). Option D is wrong because self-aware AI is purely fictional.

Question 2

A team of engineers is building an AI-powered drone that sprays pesticide only on infected crops. The drone uses cameras to identify diseased plants and avoids healthy ones. Which application domain does this belong to?

- A) Smart Cities
- B) Healthcare
- C) Agriculture
- D) Finance

Correct Answer: C

Explanation: Crop disease detection and targeted pesticide spraying is clearly an agriculture application. Smart Cities (A) involves traffic, waste, energy. Healthcare (B) involves patients and hospitals. Finance (D) involves money and transactions.

Question 3

A hospital deploys an AI that analyzes chest X-rays to detect pneumonia. The AI was trained on 100,000 labeled X-ray images. After deployment, doctors notice the AI performs poorly on X-rays from elderly patients. What lifecycle step was likely skipped or done poorly?

- A) Model Deployment
- B) Data Collection and Preprocessing
- C) Problem Definition
- D) Monitoring

Correct Answer: B

Explanation: The AI fails on elderly patients because the training data probably did not include enough X-rays from elderly people. This is a data quality/collection issue. While monitoring (D) helps discover the problem, the ROOT cause is poor data collection. Problem definition (C) was likely fine (detect pneumonia), and deployment (A) was successful since the system is running.

Question 4

A student says: “ChatGPT can write essays, solve math problems, and give medical advice. Therefore, it is Artificial General Intelligence.” Is the student correct?

- A) Yes, because AGI means AI that can do many tasks.
- B) No, because ChatGPT is still limited to text-based tasks and lacks true understanding.
- C) Yes, because ChatGPT passed the Turing Test.
- D) No, because ChatGPT is a reactive machine.

Correct Answer: B

Explanation: ChatGPT is Narrow AI. It only handles text tasks and does not truly “understand” like humans. It cannot physically interact with the world, feel emotions, or transfer knowledge across completely different domains. Option A is wrong because doing many text tasks is not AGI. Option C is wrong — passing Turing Test is not the same as AGI. Option D is wrong — ChatGPT uses limited memory (it remembers conversation context), not just reactive processing.

Question 5

A manufacturing company wants to predict when its machines will break down so they can repair them before failure. Which AI application best describes this?

- A) Quality Control
- B) Predictive Maintenance
- C) Algorithmic Trading
- D) Supply Chain Optimization

Correct Answer: B

Explanation: Predictive maintenance means predicting failures BEFORE they happen. Quality control (A) checks if products are good or bad. Algorithmic trading (C) is finance. Supply chain optimization (D) manages inventory and logistics.

Question 6

In the AI vs ML vs DL vs Data Science hierarchy, which statement is TRUE?

- A) Deep Learning is a superset of Machine Learning.
- B) Data Science is a subset of Artificial Intelligence.
- C) Machine Learning is a subset of Artificial Intelligence.
- D) Artificial Intelligence is a subset of Data Science.

Correct Answer: C

Explanation: ML is indeed a subset of AI. Option A is wrong because DL is a subset of ML, not a superset. Option B is wrong because Data Science is broader and includes statistics, visualization, etc., not just AI. Option D reverses the relationship.

Question 7

A city government installs AI-powered cameras at traffic signals. The AI counts vehicles and automatically adjusts green light duration. Which type of AI agent functionality does this system demonstrate?

- A) Theory of Mind
- B) Self-Aware AI
- C) Limited Memory
- D) Reactive Machine

Correct Answer: C

Explanation: The system remembers traffic counts from the last few minutes to adjust signals — this is Limited Memory. A reactive machine (D) would only look at the current second and not remember past traffic. Theory of Mind (A) and Self-Aware (B) do not exist yet.

Question 8

A startup is building an AI tutor for school students. The AI gives different math problems based on each student's weak areas. Which AI application domain is this?

- A) Healthcare
- B) Smart Cities
- C) Education
- D) Robotics

Correct Answer: C

Explanation: Personalized learning and adaptive tutoring are education applications. The other options are clearly unrelated.

Question 9

Which of the following is an example of a Reactive Machine?

- A) A self-driving car that remembers the position of nearby vehicles
- B) IBM's Deep Blue chess program that only considers the current board state
- C) A chatbot that remembers your previous messages
- D) A recommendation system that learns from your past purchases

Correct Answer: B

Explanation: Deep Blue looked at the current chess board and calculated the best move. It did NOT remember past games or learn from experience — pure reactive. All other options involve memory or learning.

Question 10

A company collects customer feedback emails, cleans them, converts text to numbers, trains a model to classify sentiment (positive/negative), tests it on new emails, and then deploys it on their website. Which step comes immediately BEFORE deployment?

- A) Data Collection
- B) Data Preprocessing
- C) Model Evaluation
- D) Problem Definition

Correct Answer: C

Explanation: The AI lifecycle order is: Problem Definition → Data Collection → Preprocessing → Training → Evaluation → Deployment → Monitoring. So evaluation comes right before deployment. If evaluation fails, you go back and fix the model before deploying.

Question 11

Which statement about the AI lifecycle is FALSE?

- A) Data preprocessing includes handling missing values and removing duplicates.

- B) Once deployed, an AI system never needs to be updated.
- C) Problem definition should clearly state what success looks like.
- D) Model evaluation uses data that the model has never seen before.

Correct Answer: B

Explanation: This statement is FALSE. AI systems MUST be monitored and updated regularly because real-world data changes (this is called “concept drift”). All other statements are true.

Question 12

A farmer uses an AI app that takes a photo of a wheat leaf and tells whether the crop is healthy or diseased. The app was trained on thousands of leaf images. Which technologies are involved? (Select the best combination)

- A) Only Data Science
- B) AI, Machine Learning, and Deep Learning
- C) Only Robotics
- D) Only Symbolic AI

Correct Answer: B

Explanation: The app uses AI (broad concept), ML (learning from data), and most likely Deep Learning (neural networks for image recognition). Data Science (A) is partially involved but not the core technology. Robotics (C) is not involved — it is a mobile app. Symbolic AI (D) uses hand-written rules, not image training.

UNIT II: AI PROBLEM SOLVING

2.1 Problem Formulation and State-Space Representation

Simple Definition

Problem formulation means describing a real-world problem in a way that a computer can understand and solve. **State-space representation** is a method to show all possible situations (states) the problem can be in.

Why It Matters

Before any AI can solve a problem, we must tell it EXACTLY what the problem is. A badly defined problem leads to a useless solution. This is the foundation of ALL AI problem-solving.

Easy Analogy

Imagine you are planning a trip from Jalandhar to Delhi.

- **Problem formulation** = Deciding your starting point, destination, mode of transport, and constraints (budget, time)
- **State-space** = All possible places you could be during the journey — Jalandhar bus stand, Ludhiana railway station, Chandigarh airport, Delhi ISBT, etc.

Detailed Explanation

What is Problem Formulation? To formulate a problem, we define four things:

1. **Initial State:** Where we start
 - Example: The 8-puzzle in its scrambled state
2. **Actions:** What moves are allowed
 - Example: In the 8-puzzle, you can slide a tile up, down, left, or right into the empty space
3. **Transition Model:** What happens when we take an action
 - Example: If I slide tile 3 left, the empty space moves to where tile 3 was
4. **Goal State:** What we want to reach
 - Example: The 8-puzzle with all tiles in order (1,2,3,4,5,6,7,8, blank)
5. **Path Cost:** What each move “costs” (time, money, distance)
 - Example: Each tile slide costs 1

What is State-Space Representation? The **state space** is the set of ALL possible states a problem can be in. We can visualize it as a graph:

- **Nodes** = States (situations)
- **Edges** = Actions (transitions between states)

Visual Description: Imagine a giant tree. The root is your starting point. Each branch is a possible action. Each leaf or node is a new state. The AI explores this tree to find the path to the goal.

Practical Example

Example 1: The 8-Puzzle Problem

- **Initial State:** 1 3 4 8 2 7 6 5
- **Actions:** Slide any tile adjacent to the blank space (up, down, left, right)
- **Goal State:** 1 2 3 4 5 6 7 8
- **Path Cost:** 1 per move

Example 2: Route Planning (Jalandhar to Delhi)

- **Initial State:** You are at Jalandhar
- **Actions:** Drive to nearby cities (Ludhiana, Amritsar, etc.)
- **Goal State:** You reach Delhi
- **Path Cost:** Distance in kilometers or time in hours

The state space includes every city you might pass through.

Common Mistakes Students Make

- **✗** Forgetting to define the goal state clearly. Without a goal, the AI wanders forever.
- **✗** Ignoring path costs. In real life, the shortest path is not always the fastest (a highway might be longer but faster).
- **✗** Making the state space too large. Some problems have millions of states, making them impossible to solve with simple methods.

Quick Summary

- Problem formulation = Initial state + Actions + Transition model + Goal state + Path cost
 - State space = All possible states shown as a graph (nodes = states, edges = actions)
 - Every AI problem must be formulated before it can be solved
 - State-space representation helps us visualize and plan the solution
-

2.2 Problem Characteristics

Simple Definition

Problem characteristics are the features of a problem that help us decide which AI technique to use.

Why It Matters

Not all problems are the same. A problem with clear rules needs a different approach than a problem with uncertainty. Knowing characteristics helps you pick the right tool.

Easy Analogy

Choosing an AI technique is like choosing a vehicle:

- Short distance in city = Scooter
- Long distance with luggage = Car
- Crossing an ocean = Ship
- You must know the “characteristics” of your journey to choose the right vehicle.

Detailed Explanation

Here are the main problem characteristics:

1. Decomposability Can the problem be broken into smaller, independent sub-problems?

- **Decomposable:** Calculating your total semester marks (add each subject separately, then sum).
- **Not Decomposable:** Solving a Rubik’s cube (every move affects the whole cube).

2. Solution Steps Ignorability Can we ignore past steps and only focus on the current state?

- **Ignorable:** Theorem proving (if step 3 is wrong, you can just delete it).
- **Non-ignorable:** Cooking (you cannot “un-mix” salt into a curry).

3. Predictability Is the outcome of every action known for sure?

- **Predictable:** Chess (if you move a pawn, you know exactly where it goes).
- **Unpredictable:** Stock market prediction (many unknown factors).

4. Solution Quality Does any solution work, or do we need the BEST solution?

- **Any solution:** Finding any route from Jalandhar to Delhi.
- **Best solution:** Finding the CHEAPEST route from Jalandhar to Delhi.

5. Knowledge Requirements How much knowledge about the world does the AI need?

- **Little knowledge:** Solving a maze (just know walls and paths).
- **Lots of knowledge:** Medical diagnosis (needs anatomy, symptoms, drug interactions).

6. Interaction with the World Does the AI act alone, or does it interact with humans or other agents?

- **Single agent:** Solving a crossword puzzle alone.
- **Multi-agent:** Playing chess against a human opponent.

Practical Example

Scenario: A robot is delivering packages in a warehouse.

Characteristic	Analysis
Decomposable?	Yes —deliver package A, then B, then C
Predictable?	Mostly yes —unless humans walk in the way
Best solution?	Yes —shortest route saves time and battery
Knowledge needed?	Map of warehouse, locations of packages
Single or multi-agent?	Could be multi-agent if many robots share the warehouse

Based on this, we might choose A* search (for best path) with collision avoidance (for multi-agent interaction).

Common Mistakes Students Make

- **✗** Assuming all problems need the “best” solution. Sometimes “good enough” is fine.
- **✗** Treating unpredictable problems as predictable. This leads to wrong AI choices.
- **✗** Not checking if the problem is decomposable. If it is, divide and conquer!

Quick Summary

- Decomposable? → Break it into smaller problems
 - Predictable? → Use search algorithms
 - Need best solution? → Use informed search (A*, etc.)
 - Lots of knowledge? → Use knowledge-based systems
 - Multi-agent? → Use game theory or multi-agent systems
-

2.3 Search Space and Solution Space

Simple Definition

- **Search space** = All possible paths the AI explores to find a solution
- **Solution space** = Only the paths that actually lead to the goal

Why It Matters

The search space is often HUGE. Smart AI tries to explore as little of it as possible while still finding the solution. Understanding this difference helps you appreciate why some algorithms are faster than others.

Easy Analogy

Imagine you lost your keys in a college campus.

- **Search space** = Every room, corridor, garden, and canteen you check
- **Solution space** = Only the places where your keys ACTUALLY are (maybe just the library and the canteen)

A smart person searches efficiently. A foolish person checks the same places again and again.

Detailed Explanation

Search Space This is the complete set of all states and actions the AI might consider. In a route-finding problem, the search space includes every road, every turn, every city.

Size matters: If the search space is too large, the AI might take forever to find the answer.

Solution Space This is the subset of the search space that actually contains valid solutions. There might be one solution or many.

Example: In the 8-puzzle, the search space has 362,880 possible states. But the solution space is just the ONE sequence of moves that solves your particular scrambled puzzle.

Why the Difference Matters A good AI algorithm:

1. Explores the search space efficiently
2. Quickly narrows down to the solution space
3. Avoids revisiting the same states (no loops)

Visual Description

Imagine a maze:

- The **search space** is the entire maze — every corridor.
- The **solution space** is only the path from entrance to exit.
- A bad algorithm wanders through the whole maze.
- A good algorithm uses clues (heuristics) to head straight toward the exit.

Practical Example

Example: Finding a Word in a Dictionary

- **Search space:** All 50,000 words in the dictionary
- **Solution space:** The one word you are looking for
- **Bad approach:** Start from page 1 and read every word (linear search)
- **Good approach:** Open the middle, decide if your word is before or after, and repeat (binary search)

Common Mistakes Students Make

- ❌ Thinking search space and solution space are the same. They are NOT.
- ❌ Believing a larger search space always means slower solving. Smart algorithms handle large spaces well.
- ❌ Forgetting that revisiting states wastes time. Good algorithms remember where they have been.

Quick Summary

- Search space = Everything the AI might explore
 - Solution space = Only the valid paths to the goal
 - Good AI = explores search space efficiently, finds solution space quickly
 - Avoiding repeated states is crucial for speed
-

2.4 Performance Measures

Simple Definition

Performance measures tell us how good an AI solution is. They are like report card grades for AI algorithms.

Why It Matters

If you build an AI, how do you know it is good? Performance measures give you numbers to compare different algorithms and choose the best one.

Easy Analogy

Imagine three friends running a race from the hostel gate to the academic block:

- **Completeness:** Did everyone finish the race?

- **Optimality:** Who took the shortest path?
- **Time Complexity:** Who finished fastest?
- **Space Complexity:** Who carried the least weight (needed least energy)?

Detailed Explanation

The four main performance measures for search algorithms are:

1. Completeness Definition: Is the algorithm GUARANTEED to find a solution if one exists?

- **Complete:** Yes, it will always find the solution.
- **Incomplete:** No, it might miss the solution.
- **Example:** BFS (Breadth First Search) is complete. It will find the goal if it exists.

2. Optimality Definition: Does the algorithm find the BEST (lowest cost) solution?

- **Optimal:** Yes, it finds the cheapest/shortest path.
- **Not optimal:** It finds A solution, but maybe not the best one.
- **Example:** A* Search is optimal (with the right heuristic). DFS is not optimal.

3. Time Complexity Definition: How long does the algorithm take to run?

- Measured in terms of the number of nodes expanded or explored
- Usually written as $O(b^d)$ where b = branching factor, d = depth of solution
- **Lower time complexity = Faster algorithm**

4. Space Complexity Definition: How much memory does the algorithm need?

- Measured in terms of the maximum number of nodes stored in memory at once
- **Lower space complexity = Less memory needed**

Practical Example

Scenario: Finding the shortest path from your hostel to the LPU main gate.

Algorithm	Complete?	Optimal?	Time	Space
BFS	Yes	Yes (if costs are equal)	High	High
DFS	Yes (in finite spaces)	No	Medium	Low
A*	Yes	Yes	Low	Medium

Analysis: If you want the shortest path and have enough memory, use BFS or A*. If memory is limited and you just need ANY path, DFS might work.

Common Mistakes Students Make

- **✗** Confusing completeness with optimality. An algorithm can find a solution (complete) but not the best one (not optimal).
- **✗** Thinking lower time complexity always means better. Sometimes you need optimality even if it takes longer.
- **✗** Ignoring space complexity. On a phone or embedded device, memory is limited.

Quick Summary

- **Completeness:** Will it find a solution if one exists?
 - **Optimality:** Will it find the BEST solution?
 - **Time Complexity:** How fast is it?
 - **Space Complexity:** How much memory does it need?
 - No algorithm is perfect in all four. You choose based on your problem.
-

2.5 Overview of AI-Based Problem-Solving

Simple Definition

AI solves problems by searching through possible solutions, using knowledge, or learning from data. The approach depends on whether the problem is well-defined, uncertain, or requires learning.

Why It Matters

This overview connects everything in Unit II. It helps you understand WHEN to use search, WHEN to use knowledge, and WHEN to use learning.

Easy Analogy

Think of AI problem-solving like three different ways to find a book in a library:

1. **Search-based:** You walk through every shelf (systematic search)
2. **Knowledge-based:** You ask the librarian who knows exactly where it is (expert knowledge)
3. **Learning-based:** You remember where you found similar books last time (experience)

Detailed Explanation

AI problem-solving has three main approaches:

Approach 1: Search-Based Problem Solving

- Used when the problem has a clear initial state, actions, and goal
- The AI explores the state space systematically
- Types: Uninformed search (no clues) and Informed search (uses hints/heuristics)
- **Best for:** Pathfinding, puzzles, games with clear rules
- **Examples:** GPS navigation, chess, 8-puzzle, maze solving

Approach 2: Knowledge-Based Problem Solving

- Used when the problem requires expert knowledge or logical reasoning
- The AI uses facts and rules stored in a knowledge base
- Types: Logic-based reasoning, rule-based systems, semantic networks
- **Best for:** Medical diagnosis, legal advice, troubleshooting
- **Examples:** Expert systems that diagnose diseases, tax calculation systems

Approach 3: Learning-Based Problem Solving

- Used when the problem is too complex to define with rules
- The AI learns patterns from data
- Types: Supervised learning, unsupervised learning, reinforcement learning
- **Best for:** Image recognition, speech recognition, recommendation systems
- **Examples:** Netflix recommendations, spam filters, face unlock

When to Use Which?

Situation	Best Approach	Why?
Clear rules, known goal	Search	Systematic exploration works
Needs expert knowledge	Knowledge-based	Rules capture human expertise
Too complex for rules, lots of data	Learning-based	Data reveals hidden patterns
Real-time decisions, uncertain environment	Learning + Search	Combine adaptability with planning

Practical Example

Scenario: Building an AI for a hospital emergency room.

- **Search:** Finding the fastest route for an ambulance through traffic
- **Knowledge-based:** An expert system that suggests initial treatment based on symptoms
- **Learning-based:** Predicting which patients are at highest risk of heart attack based on their vital signs history

A complete hospital AI would use ALL THREE approaches together.

Common Mistakes Students Make

- **✗** Using search for problems that need learning. If the problem has millions of rules you cannot write, use ML.
- **✗** Using learning for simple, well-defined problems. A calculator does not need machine learning.
- **✗** Forgetting that real-world AI systems often combine all three approaches.

Quick Summary

- Search-based = Explore state space systematically (puzzles, pathfinding)
- Knowledge-based = Use rules and logic (diagnosis, advice)

- Learning-based = Learn from data (recognition, prediction)
 - Best AI systems often combine all three
-

UNIT II: KEY POINTS TO REMEMBER

- Problem formulation needs: Initial state, Actions, Transition model, Goal state, Path cost
 - State-space representation shows all possible states as a graph (nodes = states, edges = actions)
 - Problem characteristics: Decomposability, predictability, solution quality, knowledge needs, single/multi-agent
 - Search space = all paths explored; Solution space = only valid paths to the goal
 - Four performance measures: Completeness, Optimality, Time complexity, Space complexity
 - No single algorithm is best in all four measures — choose based on your needs
 - AI problem-solving has three approaches: Search-based, Knowledge-based, Learning-based
 - Real-world AI often combines all three approaches
-

UNIT II: SELF-CHECK QUESTIONS

1. What are the five components of problem formulation?
 2. Give an example of a decomposable problem and a non-decomposable problem.
 3. What is the difference between search space and solution space?
 4. Define completeness and optimality in your own words.
 5. Why might DFS be preferred over BFS in some situations?
 6. Name three approaches to AI problem-solving and give one example of each.
 7. What does it mean if an algorithm has $O(b^d)$ time complexity?
 8. When would you choose a knowledge-based approach over a learning-based approach?
-

UNIT II: SCENARIO-BASED MCQs

Question 1

A student is solving a Rubik's cube using an AI algorithm. The algorithm must find the shortest sequence of moves to solve the cube. Which performance measure is MOST important here?

- A) Space Complexity
- B) Optimality
- C) Completeness only
- D) Time Complexity only

Correct Answer: B

Explanation: The goal is the SHORTEST sequence of moves, so optimality (finding the best/cheapest solution) is most important. While completeness (C) and time complexity (D) matter, they are secondary

to finding the optimal solution. Space complexity (A) is the least important for a puzzle on a computer with ample memory.

Question 2

An AI is designed to find a path for a delivery robot in a warehouse. The warehouse map is fixed, and the robot must reach the delivery point. The team wants to guarantee the robot always finds the goal if it exists. Which property are they focusing on?

- A) Optimality
- B) Completeness
- C) Time Complexity
- D) Decomposability

Correct Answer: B

Explanation: “Guarantee the robot always finds the goal if it exists” is the definition of completeness. Optimality (A) would mean finding the shortest path. Time complexity (C) is about speed. Decomposability (D) is about breaking the problem into sub-problems, which is unrelated.

Question 3

A navigation app like Google Maps uses AI to find the best route. It considers distance, traffic, and road type. Before the algorithm runs, the developers must define the starting location, destination, possible roads, and travel time. What is this process called?

- A) State-Space Search
- B) Problem Formulation
- C) Heuristic Evaluation
- D) Performance Measurement

Correct Answer: B

Explanation: Defining the initial state (starting location), goal state (destination), actions (possible roads), and path cost (travel time) is problem formulation. State-space search (A) comes AFTER formulation. Heuristic evaluation (C) is used during informed search. Performance measurement (D) evaluates the solution after finding it.

Question 4

In an 8-puzzle game, the blank tile can move up, down, left, or right. All possible arrangements of the tiles represent the _____, while only the sequence of moves that

actually solves the puzzle represents the _____.

- A) Search space; Solution space
- B) Solution space; Search space
- C) State space; Action space
- D) Action space; State space

Correct Answer: A

Explanation: All possible arrangements = search space. The actual path to the goal = solution space. The other options reverse the terms or introduce incorrect terms.

Question 5

A bank wants to build an AI that approves or rejects loan applications. The problem has thousands of rules about income, credit score, employment history, and existing loans. The rules change frequently based on government regulations. Which problem-solving approach is MOST suitable?

- A) Search-based
- B) Knowledge-based
- C) Learning-based only
- D) Game-based

Correct Answer: B

Explanation: This problem has clear, changing rules and requires expert knowledge (banking regulations). A knowledge-based system (rules engine) is ideal because rules can be updated easily. Search-based (A) is for pathfinding. Learning-based (C) could help but the frequent rule changes make a knowledge base more practical. Game-based (D) is irrelevant.

Question 6

A team is building an AI to play chess. The game has clear rules, a known initial state, and a defined goal (checkmate). However, the number of possible game states is astronomically large (about 10^{120}). Which problem characteristic does this illustrate?

- A) The problem is non-decomposable
- B) The search space is extremely large
- C) The problem is unpredictable

D) The solution is non-ignorable

Correct Answer: B

Explanation: 10^{120} possible states means the search space is unimaginably large. This is why chess AI uses smart heuristics (like Alpha-Beta pruning) instead of brute force. The problem IS decomposable in strategy (A is wrong), it IS predictable (C is wrong), and solution steps ARE ignorable (D is wrong).

Question 7

A factory wants an AI to predict machine failures. The AI learns from past breakdown data, temperature readings, and vibration patterns. It does not use hand-written rules. Which approach is this?

- A) Search-based
- B) Knowledge-based
- C) Learning-based
- D) Logic-based

Correct Answer: C

Explanation: The AI learns from data (past breakdowns, sensor readings) without explicit rules. This is the definition of learning-based (Machine Learning). Knowledge-based (B) and logic-based (D) require explicit rules. Search-based (A) explores states, which is not what is happening here.

Question 8

A student is comparing two algorithms for solving a maze. Algorithm X always finds the shortest path but uses a lot of memory. Algorithm Y uses very little memory but sometimes finds a longer path. Which statement is TRUE?

- A) Algorithm X is optimal but has higher space complexity.
- B) Algorithm Y is complete but has higher time complexity.
- C) Algorithm X is incomplete but has lower time complexity.
- D) Algorithm Y is optimal and has lower space complexity.

Correct Answer: A

Explanation: Algorithm X finds the shortest path = optimal. Uses a lot of memory = higher space complexity. Algorithm Y uses little memory = lower space complexity, but finds longer paths = NOT optimal. Option B is wrong because we do not know if Y is complete. Option C is wrong because X is complete (finds the path) and we do not know its time complexity relative to Y. Option D is wrong because Y is explicitly NOT optimal.

Question 9

An AI is solving a problem where the outcome of each action is completely known in advance. For example, moving a chess piece always results in a predictable board state. What is this characteristic called?

- A) Decomposability
- B) Predictability
- C) Determinism
- D) Optimality

Correct Answer: B

Explanation: This is predictability — the outcome of actions is known. In AI terminology, this is also called a “deterministic” environment, but “predictability” is the problem characteristic taught in this context. Decomposability (A) is about breaking into sub-problems. Optimality (D) is about solution quality.

Question 10

A company needs an AI that can solve a complex scheduling problem by breaking it into smaller scheduling problems for individual departments, solving each separately, and then combining the results. Which characteristic makes this possible?

- A) Predictability
- B) Decomposability
- C) Ignorability
- D) Completeness

Correct Answer: B

Explanation: Breaking a problem into smaller, independent sub-problems is decomposability. Predictability (A) is about known outcomes. Ignorability (C) is about whether past steps matter. Completeness (D) is an algorithm property, not a problem characteristic.

Question 11

In a water jug problem, you have a 4-liter jug and a 3-liter jug. You need exactly 2 liters in the 4-liter jug. Each state represents how much water is in each jug. The set of all possible water levels in both jugs is called the:

- A) Solution space

- B) Search space
- C) Action space
- D) Goal space

Correct Answer: B

Explanation: All possible water levels in both jugs = all possible states = search space. The solution space would be only the sequence of pour/fill/empty actions that gives exactly 2 liters in the 4-liter jug.

Question 12

A university exam asks students to identify which AI approach is best for a given scenario. The key skill being tested is:

- A) Memorizing definitions
- B) Matching problem characteristics to AI approaches
- C) Writing code for every algorithm
- D) Knowing the history of AI

Correct Answer: B

Explanation: Scenario-based questions test whether you can analyze a situation (its characteristics) and match it to the right AI approach. This is exactly what LPU CA questions do. Memorizing definitions (A) is not enough. Writing code (C) and history (D) are not the focus of these questions.

UNIT III: PROBLEM SOLVING USING SEARCH TECHNIQUES

3.1 Uninformed Search: Breadth First Search (BFS) and Depth First Search (DFS)

Simple Definition

Uninformed search means the AI explores the state space WITHOUT any clues about which direction is better. It blindly searches until it finds the goal. **BFS** explores level by level. **DFS** explores one path deeply before backtracking.

Why It Matters

BFS and DFS are the building blocks of all search algorithms. Understanding them helps you understand why smarter algorithms were invented.

Easy Analogy

Imagine you lost your friend in a multi-floor shopping mall.

- **BFS (Breadth First Search):** You check EVERY shop on Floor 1, then EVERY shop on Floor 2, then Floor 3, and so on. You explore “wide” before going “deep.”
- **DFS (Depth First Search):** You enter the first shop on Floor 1, go into its storeroom, then the back exit, then the staircase, and keep going as deep as possible. If you hit a dead end, you come back and try another path.

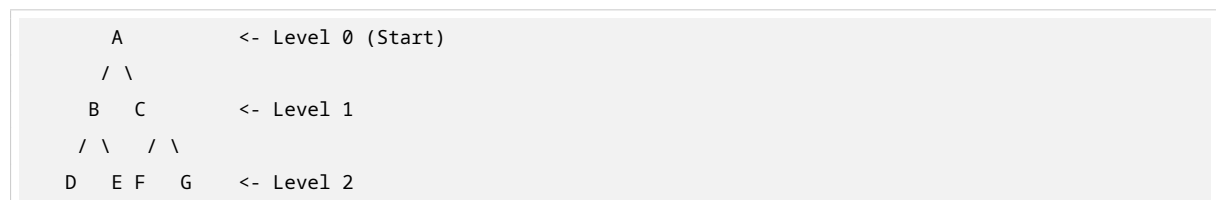
Detailed Explanation

Breadth First Search (BFS) How it works (step by step):

1. Start at the initial state (root node)
2. Explore ALL neighbors of the current node first
3. Then explore neighbors of those neighbors
4. Continue level by level until the goal is found

Data Structure Used: Queue (First In, First Out — like a line at the canteen)

Visual Description: Imagine a family tree. BFS checks the grandfather, then all his children, then all his grandchildren, then all great-grandchildren — level by level.



BFS order: A -> B -> C -> D -> E -> F -> G

Advantages:

- Complete (always finds the goal if it exists)
- Optimal for unweighted graphs (finds the shortest path in terms of number of steps)

Disadvantages:

- High memory usage (stores all nodes at the current level)
- Can be slow for deep goals

Depth First Search (DFS) How it works (step by step):

1. Start at the initial state (root node)
2. Pick ONE neighbor and explore it fully
3. Keep going deeper until you hit the goal or a dead end

4. If dead end, backtrack to the last node with unexplored neighbors
5. Repeat until goal is found or all paths are exhausted

Data Structure Used: Stack (Last In, First Out — like a stack of plates) or recursion

Visual Description: Using the same family tree: DFS order: A -> B -> D -> E -> C -> F -> G (or A -> B -> D, backtrack, B -> E, backtrack, A -> C -> F, backtrack, C -> G)

Advantages:

- Low memory usage (only stores the current path)
- Can find deep goals quickly if lucky

Disadvantages:

- Not optimal (might find a long path when a short one exists)
- Can get stuck in infinite loops in infinite spaces (not complete in infinite graphs)

Practical Example

Example: Maze Solving

Maze layout:

```
Start -> A -> B -> Goal
|
C -> D
```

BFS explores: Start -> A -> C -> B -> D -> Goal

- Finds Goal in 3 steps (Start -> A -> B -> Goal)
- But it also explored C and D unnecessarily

DFS explores: Start -> A -> B -> Goal

- Might find Goal faster (if lucky)
- But if it went Start -> C -> D first, it would take longer

Comparison Table

Feature	BFS	DFS
Strategy	Level by level	Deep first, then backtrack
Data Structure	Queue	Stack/Recursion
Complete?	Yes	Yes (in finite spaces)
Optimal?	Yes (unweighted)	No
Time Complexity	$O(b^d)$	$O(b^m)$
Space Complexity	$O(b^d)$	$O(bm)$
Best For	Shortest path, shallow goals	Deep goals, memory constraints

b = branching factor, d = depth of solution, m = maximum depth

Common Mistakes Students Make

- ❌ Thinking DFS is always faster. It depends on where the goal is.
- ❌ Forgetting BFS needs a queue and DFS needs a stack.
- ❌ Believing BFS is always better. If memory is limited, DFS might be your only option.
- ❌ Not understanding that BFS finds the shortest path in unweighted graphs.

Quick Summary

- Uninformed search = no clues, just systematic exploration
 - BFS = level by level, uses queue, complete and optimal, high memory
 - DFS = deep first, uses stack, complete (finite), not optimal, low memory
 - Choose BFS for shortest path; choose DFS when memory is limited
-

3.2 Informed Search: Best First Search, Hill Climbing, and A* Search

Simple Definition

Informed search means the AI uses a **heuristic** (a hint or estimate) to decide which path looks most promising. It does not explore blindly — it has a “gut feeling” about which way to go.

Why It Matters

Informed search is much faster than uninformed search for large problems. It is what makes GPS navigation, game AI, and robot pathfinding practical.

Easy Analogy

Imagine you are in a dark forest trying to find a village.

- **Uninformed search:** You try every path randomly.
- **Informed search:** You see a light in the distance and walk toward it. The light is your heuristic — it gives you a hint about where the village is.

Detailed Explanation

Best First Search (Greedy Best First Search) How it works:

1. At every step, choose the node that looks CLOSEST to the goal
2. Use a heuristic function $h(n)$ = estimated cost from node n to goal
3. Always expand the node with the LOWEST $h(n)$

Visual Description: Like a greedy person always choosing the path that seems closest to the destination, even if it might lead to a longer overall route.

Advantages:

- Fast — focuses search toward the goal
- Good when the heuristic is accurate

Disadvantages:

- NOT optimal — can get trapped in local optima
- NOT complete — might miss the goal if the heuristic is misleading

Hill Climbing How it works:

1. Start at the initial state
2. Look at all neighboring states
3. Move to the neighbor that improves your position the MOST
4. Repeat until no neighbor is better than current state

Visual Description: Imagine climbing a mountain in fog. You can only see a few meters ahead. You always walk uphill. If you reach a small hill (local maximum), you stop — even if the real mountain peak is on the other side.

Types of Hill Climbing:

- **Simple Hill Climbing:** Pick the FIRST better neighbor
- **Steepest Ascent Hill Climbing:** Pick the BEST neighbor
- **Stochastic Hill Climbing:** Pick a random better neighbor

Problems with Hill Climbing:

- **Local Maximum:** A peak that is not the highest peak. The algorithm stops because no neighbor is better.
- **Plateau:** A flat area where all neighbors are equally good. The algorithm wanders aimlessly.
- **Ridge:** A narrow path uphill. The algorithm might miss it because it only looks at immediate neighbors.

Advantages:

- Very simple to implement
- Low memory usage

Disadvantages:

- NOT complete — can get stuck
- NOT optimal — finds local maxima, not global maxima

A* Search (A-Star Search) How it works:

1. Uses TWO pieces of information:
 - $g(n)$ = actual cost from start to node n
 - $h(n)$ = estimated cost from node n to goal
2. Calculate $f(n) = g(n) + h(n)$
3. Always expand the node with the LOWEST $f(n)$

Why A* is special:

- It balances what we ALREADY spent ($g(n)$) with what we EXPECT to spend ($h(n)$)

- If the heuristic $h(n)$ is **admissible** (never overestimates), A^* is both **complete** and **optimal**

Visual Description: Imagine planning a road trip. $g(n)$ is the fuel you have already used. $h(n)$ is your GPS estimate of remaining fuel needed. $f(n)$ is your total estimated fuel. A^* always picks the route with the lowest total estimated fuel.

Advantages:

- Complete (finds goal if it exists)
- Optimal (with admissible heuristic)
- Efficient — explores fewer nodes than BFS

Disadvantages:

- Needs a good heuristic — bad heuristic = bad performance
- Higher memory usage than DFS

Comparison Table

Feature	Best First	Hill Climbing	A^* Search
Uses heuristic?	Yes	Yes	Yes
Considers path cost so far?	No	No	Yes ($g(n)$)
Formula	$h(n)$ only	$h(n)$ only	$f(n) = g(n) + h(n)$
Complete?	No	No	Yes
Optimal?	No	No	Yes (if h is admissible)
Best For	Quick approximate solutions	Simple optimization	Best pathfinding

Practical Example

Example: 8-Puzzle with A^*

Heuristic $h(n)$ = number of tiles out of place (Manhattan distance is better, but let's keep it simple).

Current state:

1	2	3
8		4
7	6	5

Goal state:

1	2	3
4	5	6
7	8	

$h(n) = 4$ (tiles 4, 5, 6, and blank are out of place)

If we moved tile 6 up: $g(\text{new}) = g(\text{current}) + 1 = 1$ $h(\text{new}) = 3$ (better!) $f(\text{new}) = 1 + 3 = 4$

A^* will explore moves with lowest $f(n)$ first.

Common Mistakes Students Make

- ❌ Thinking Best First Search is optimal. It is NOT — it is greedy.
- ❌ Confusing Hill Climbing with gradient descent. Hill climbing is for discrete states; gradient descent is for continuous functions.
- ❌ Forgetting that A^* needs an admissible heuristic to be optimal. If $h(n)$ overestimates, A^* is not guaranteed optimal.
- ❌ Believing informed search is always better. For small problems, BFS might be simpler and just as fast.

Quick Summary

- Informed search uses heuristics (hints) to guide exploration
 - Best First = greedy, only looks at estimated remaining cost $h(n)$
 - Hill Climbing = always move to the best neighbor, can get stuck in local maxima
 - A^* = combines actual cost $g(n)$ + estimated cost $h(n)$, both complete and optimal
 - A^* is the gold standard for pathfinding when you have a good heuristic
-

3.3 Heuristic Functions

Simple Definition

A **heuristic function** is a rule of thumb or educated guess that estimates how close a state is to the goal. It helps the AI make smart choices without calculating everything.

Why It Matters

Without heuristics, AI would explore every possible path — too slow for real-world problems. A good heuristic makes AI fast. A bad heuristic makes AI wrong or slow.

Easy Analogy

Imagine you are playing hide and seek. Your friend gives you a hint: “I am somewhere in the north wing of the building.” That hint is a heuristic. It is not exact, but it narrows your search. A bad hint (“I am on Earth somewhere”) is useless. A good hint (“I am in room 302”) is very helpful.

Detailed Explanation

What is a Heuristic? A heuristic is a function $h(n)$ that takes a state n and returns an estimate of the cost to reach the goal from n .

Key Properties:

1. **Admissible Heuristic:** $h(n)$ NEVER overestimates the true cost. It is optimistic.
 - Example: Straight-line distance on a map never overestimates actual road distance.
 - If h is admissible, A^* is guaranteed optimal.
2. **Consistent (Monotonic) Heuristic:** The estimated cost never decreases by more than the actual step cost.
 - $h(n) \leq \text{cost}(n, n') + h(n')$ for all neighbors n'
 - Consistency implies admissibility.

Common Heuristics For Route Finding:

- **Straight-line distance (Euclidean distance):** The “as the crow flies” distance between two points. Always admissible because the road can only be longer, never shorter.

For 8-Puzzle:

- **Misplaced Tiles:** Count how many tiles are not in their goal position. Admissible because each misplaced tile needs at least one move.
- **Manhattan Distance:** Sum of horizontal + vertical distances each tile is from its goal position. More accurate than misplaced tiles.

For Maze Solving:

- **Distance to goal:** Euclidean or Manhattan distance from current position to goal.

Practical Example

Example: GPS Navigation

You want to drive from Jalandhar to Delhi.

- Actual road distance might be 380 km.
- Straight-line (heuristic) distance is 340 km.
- The heuristic (340) is LESS than actual cost (380), so it is **admissible**.

If your heuristic said “500 km,” it would overestimate and NOT be admissible. A* might then miss the optimal route.

How to Design a Good Heuristic

1. It should be **fast to calculate** — otherwise you lose the time saved by searching less
2. It should be **as accurate as possible** — closer to true cost means better pruning
3. It should be **admissible** — for A* to be optimal

Common Mistakes Students Make

- **✗** Thinking any guess is a good heuristic. It must be admissible for A* optimality.
- **✗** Using a heuristic that is too expensive to compute. The time saved in search might be lost in calculating $h(n)$.
- **✗** Confusing admissibility with consistency. All consistent heuristics are admissible, but not vice versa.

Quick Summary

- Heuristic = educated guess about remaining cost to goal
- Admissible heuristic = never overestimates (essential for A* optimality)
- Common heuristics: straight-line distance, misplaced tiles, Manhattan distance
- Good heuristic = fast to compute + accurate + admissible

3.4 Comparison and Applications of Search Algorithms

Simple Definition

Different search algorithms have different strengths and weaknesses. Choosing the right one depends on your problem’s size, structure, and requirements.

Why It Matters

In real life, you cannot try every algorithm. You must pick the right tool for the job. This section helps you make that choice.

Easy Analogy

Choosing a search algorithm is like choosing a mode of transport:

- **BFS** = Auto-rickshaw — reliable, finds shortest route, but slow in heavy traffic (high memory)
- **DFS** = Motorcycle — fast, uses little space, but might take a long scenic route (not optimal)
- **A*** = GPS with live traffic — smart, finds best route, but needs good signal (good heuristic)
- **Hill Climbing** = Walking uphill in fog — simple, but you might get stuck on a small hill

Detailed Comparison

Algorithm	Type	Complete?	Optimal?	Time	Space	Best Use Case
BFS	Uninformed	Yes	Yes (unweighted)	$O(b^d)$	$O(b^d)$	Shortest path, small state space
DFS	Uninformed	Yes (finite)	No	$O(b^m)$	$O(bm)$	Deep goals, memory limits
Best First	Informed	No	No	$O(b^m)$	$O(b^m)$	Quick approximate solution
Hill Climbing	Informed	No	No	$O(\text{infinite})$	$O(1)$	Simple optimization, small problems
A*	Informed	Yes	Yes	$O(b^d)$	$O(b^d)$	Pathfinding, route planning, games

Real-World Applications

BFS Applications:

- Social network friend suggestions (find shortest connection)
- Web crawlers (explore web pages level by level)
- GPS shortest path (when all roads have equal cost)

DFS Applications:

- Maze solving
- Topological sorting (scheduling tasks with dependencies)
- Detecting cycles in graphs

- Solving puzzles with deep solutions (like Sudoku)

A* Applications:

- Google Maps / GPS navigation
- Video game pathfinding (enemies finding the player)
- Robot motion planning
- Puzzle solving (8-puzzle, 15-puzzle)

Hill Climbing Applications:

- Scheduling problems (class timetables, exam schedules)
- Traveling Salesman Problem (approximate solutions)
- Feature selection in Machine Learning

Practical Example

Scenario: You are building a game where an enemy AI must chase the player through a dungeon.

- **Option 1: BFS** — Guarantees shortest path but might lag if the dungeon is huge.
- **Option 2: DFS** — Fast but the enemy might take a ridiculously long route.
- **Option 3: A*** — Best choice. Uses distance to player as heuristic. Finds shortest path efficiently.
- **Option 4: Hill Climbing** — Enemy might get stuck behind a wall because it only looks at immediate distance.

Best choice: A*

Common Mistakes Students Make

- **✗** Using A* without a good heuristic. If you cannot define $h(n)$, BFS might be better.
- **✗** Using Hill Climbing for pathfinding. It is designed for optimization, not pathfinding.
- **✗** Forgetting that BFS and A* have high memory needs. For mobile games with limited RAM, you might need iterative deepening DFS.

Quick Summary

- BFS = reliable but memory-hungry, best for shortest path
- DFS = memory-efficient but not optimal, best for deep exploration
- A* = best overall for pathfinding when you have a good heuristic
- Hill Climbing = simple but gets stuck, best for small optimization problems
- Always match the algorithm to your problem's needs

UNIT III: KEY POINTS TO REMEMBER

- Uninformed search explores without clues; informed search uses heuristics
- BFS uses a queue, explores level by level, is complete and optimal for unweighted graphs
- DFS uses a stack, explores deep first, is complete in finite spaces but not optimal
- Best First Search is greedy — only considers $h(n)$, not optimal

- Hill Climbing always moves to the best neighbor but gets stuck in local maxima, plateaus, and ridges
 - A* uses $f(n) = g(n) + h(n)$ and is both complete and optimal if $h(n)$ is admissible
 - Admissible heuristic = never overestimates true cost to goal
 - Choose BFS for small shortest-path problems, DFS for memory-limited deep search, A* for route planning and games
-

UNIT III: SELF-CHECK QUESTIONS

1. What data structure does BFS use, and why?
 2. Why is DFS not optimal? Give an example.
 3. What is the difference between Best First Search and A* Search?
 4. Define “admissible heuristic” in your own words.
 5. What are the three problems that Hill Climbing can get stuck in?
 6. When would you prefer DFS over BFS?
 7. What makes A* both complete and optimal?
 8. Give one real-world application each for BFS, DFS, and A*.
-

UNIT III: SCENARIO-BASED MCQs

Question 1

A game developer is creating an enemy AI that must find the shortest path to the player in a large maze. The maze has walls, open spaces, and the enemy can move in four directions. The developer wants the enemy to reach the player as quickly as possible without taking unnecessary detours. Which algorithm should they use?

- A) Depth First Search
- B) Breadth First Search
- C) A* Search
- D) Hill Climbing

Correct Answer: C

Explanation: A* Search is the best choice for pathfinding in games because it finds the shortest path efficiently using a heuristic (distance to player). BFS (B) would also find the shortest path but would explore too many nodes in a large maze, causing lag. DFS (A) might find a very long path. Hill Climbing (D) could get stuck behind walls because it only looks at immediate improvement.

Question 2

A robot vacuum cleaner must clean a room by visiting every spot. The team notices the robot keeps revisiting the same spots and never finishes. Which algorithmic issue is MOST likely occurring?

- A) The heuristic is not admissible
- B) The search is not complete
- C) The robot is using DFS without cycle detection
- D) The robot is using BFS with a bad heuristic

Correct Answer: C

Explanation: DFS without remembering visited states will keep going in circles (cycles). The robot revisits spots because it does not track where it has already been. Cycle detection fixes this. Option A is about heuristics, which DFS does not use. Option B is partially true but the specific symptom (revisiting) points to lack of cycle detection. Option D is wrong because BFS does not cause cycles, and BFS does not use heuristics in uninformed search.

Question 3

In an 8-puzzle solver, the heuristic function counts the number of tiles that are NOT in their correct position. This heuristic is:

- A) Not admissible because it ignores the blank tile
- B) Admissible because each misplaced tile needs at least one move
- C) Consistent but not admissible
- D) Only useful for Best First Search, not A*

Correct Answer: B

Explanation: The “misplaced tiles” heuristic is admissible because to fix a misplaced tile, you need AT LEAST one move. So the heuristic never overestimates. Option A is wrong — the blank tile does not matter. Option C is wrong — it is both admissible and consistent. Option D is wrong — it is very useful for A*.

Question 4

A delivery company wants to find the fastest route for a truck that must visit 50 cities and return to the starting point. The problem is too large for exact solutions. They need a quick, approximate solution. Which approach is most suitable?

- A) Breadth First Search
- B) A* Search with Manhattan distance
- C) Hill Climbing
- D) Depth First Search

Correct Answer: C

Explanation: This is the Traveling Salesman Problem (TSP) with 50 cities — the search space is enormous. Exact algorithms like BFS or A* would take too long. Hill Climbing can find a good (not perfect) solution quickly by iteratively improving the route. DFS (D) would be random and terrible. A* (B) with Manhattan distance does not apply to TSP.

Question 5

A student implements A* Search for maze solving but uses a heuristic that estimates the remaining distance by multiplying the straight-line distance by 2. What is the likely outcome?

- A) The algorithm will be faster but might not find the optimal path
- B) The algorithm will be slower but guaranteed optimal
- C) The algorithm will crash
- D) The algorithm will find the solution in fewer nodes than with an admissible heuristic

Correct Answer: A

Explanation: Multiplying by 2 makes the heuristic INADMISSIBLE (it can overestimate). A* with an inadmissible heuristic is NOT guaranteed to be optimal. It might still find A path, but not necessarily the shortest one. It may explore fewer nodes (faster) but at the cost of optimality. Option B is wrong because inadmissible heuristics break optimality guarantees. Option C is wrong — the algorithm does not crash. Option D is misleading because while it might explore fewer nodes, the key issue is losing optimality.

Question 6

In a web crawler that explores websites starting from a homepage, which search strategy is most natural for ensuring all linked pages at depth 1 are explored before depth 2?

- A) Depth First Search
- B) Hill Climbing
- C) Breadth First Search
- D) Best First Search

Correct Answer: C

Explanation: BFS explores all nodes at depth 1 before depth 2, depth 2 before depth 3, and so on. This is exactly how web crawlers work — they explore all links on the current page before following links to deeper pages. DFS (A) would follow one link chain deep first. Hill Climbing (B) and Best First (D) are not standard for web crawling.

Question 7

A puzzle solver using Hill Climbing reaches a state where no neighboring move improves the score, but the puzzle is not solved. This situation is called a:

- A) Plateau
- B) Ridge
- C) Local Maximum
- D) Global Maximum

Correct Answer: C

Explanation: A local maximum is a state better than all neighbors but not the best overall solution. The algorithm stops because it cannot see any improvement nearby. A plateau (A) is where all neighbors are equally good. A ridge (B) is a narrow uphill path. Global maximum (D) is the actual best solution.

Question 8

Which search algorithm would you choose if you have VERY limited memory (like a small microcontroller) and just need to find whether a solution EXISTS, not necessarily the best one?

- A) A* Search
- B) Breadth First Search
- C) Depth First Search
- D) Best First Search

Correct Answer: C

Explanation: DFS uses $O(bm)$ memory — only the current path needs to be stored. This makes it ideal for memory-constrained devices. BFS (B) and A* (A) need $O(b^d)$ memory, which might not fit. Best First (D) also needs significant memory to store the open list.

Question 9

The formula $f(n) = g(n) + h(n)$ is used in which algorithm? What do $g(n)$ and $h(n)$ represent?

- A) Best First Search; $g(n)$ = goal estimate, $h(n)$ = heuristic
- B) A* Search; $g(n)$ = cost from start, $h(n)$ = estimated cost to goal

C) Hill Climbing; $g(n)$ = gradient, $h(n)$ = height

D) BFS; $g(n)$ = depth, $h(n)$ = branching factor

Correct Answer: B

Explanation: A* Search uses $f(n) = g(n) + h(n)$, where $g(n)$ is the actual cost from the start node to n , and $h(n)$ is the heuristic estimate from n to the goal. Best First (A) uses only $h(n)$. Hill Climbing (C) does not use this formula. BFS (D) does not use heuristics at all.

Question 10

A GPS navigation system uses the straight-line distance between two points as a heuristic for finding driving directions. Why is this heuristic admissible?

A) Because the straight line is always longer than the road distance

B) Because the road distance is always greater than or equal to the straight-line distance

C) Because GPS systems never make mistakes

D) Because straight-line distance is easy to calculate

Correct Answer: B

Explanation: The road cannot be shorter than the straight line between two points (you cannot drive through buildings!). So straight-line distance $\leq$ actual road distance. This means the heuristic never OVERESTIMATES, making it admissible. Option A reverses the relationship. Option C is irrelevant to admissibility. Option D is true but not the reason for admissibility.

Question 11

In a search tree with branching factor $b = 3$ and solution depth $d = 5$, BFS might need to explore up to $3^5 = 243$ nodes in the worst case. This illustrates that BFS has:

A) Low time complexity

B) Exponential time complexity

C) Linear time complexity

D) Constant time complexity

Correct Answer: B

Explanation: $O(b^d)$ is exponential complexity. As depth increases, the number of nodes explodes. This is why BFS is impractical for very deep or wide search spaces. Option A is wrong — exponential is not low. Option C (linear) would be $O(d)$. Option D (constant) would be $O(1)$.

Question 12

A search algorithm is called “complete” if:

- A) It always finds the optimal solution
- B) It always finds a solution if one exists
- C) It uses the least memory possible
- D) It runs in polynomial time

Correct Answer: B

Explanation: Completeness means the algorithm is GUARANTEED to find a solution if one exists. Optimality (A) is a separate property. Memory usage (C) relates to space complexity. Polynomial time (D) is about efficiency, not completeness.

UNIT IV: KNOWLEDGE REPRESENTATION AND REASONING

4.1 Knowledge Representation

Simple Definition

Knowledge Representation is the way we store information, facts, and rules in a computer so that the AI can use them to think and make decisions.

Why It Matters

An AI without knowledge is like a student who has never opened a book. Knowledge representation gives the AI something to “think” with. It is the foundation of expert systems, reasoning, and intelligent agents.

Easy Analogy

Think of knowledge representation like organizing your study notes:

- Some students make bullet points (facts)
- Some draw mind maps (connections between ideas)
- Some write formulas separately
- Each method helps them “reason” about the subject during exams

Similarly, AI uses different formats to store knowledge depending on what it needs to do.

Detailed Explanation

What is Knowledge? Knowledge is not just raw data. It is:

- **Facts:** “Water boils at 100°C”
- **Rules:** “If fever AND cough, then possible flu”
- **Relationships:** “A dog is a mammal”
- **Procedures:** “To make tea: boil water, add tea leaves, add milk, sugar”

Types of Knowledge

1. **Declarative Knowledge:** “What” — facts and statements
 - Example: “The capital of France is Paris.”
2. **Procedural Knowledge:** “How” — steps and methods
 - Example: “To sort a list, compare adjacent elements and swap if out of order.”
3. **Meta-Knowledge:** Knowledge about knowledge
 - Example: “I am 90% confident that Paris is the capital of France.”

Requirements of a Good Knowledge Representation

1. **Representational Adequacy:** Can it represent ALL the knowledge needed?
2. **Inferential Adequacy:** Can the AI draw new conclusions from it?
3. **Inferential Efficiency:** Can it draw conclusions quickly?
4. **Acquisitional Efficiency:** Can new knowledge be added easily?

Practical Example

Example: Medical Diagnosis System

The AI needs to know:

- **Facts:** “Fever is a symptom. Cough is a symptom.”
- **Rules:** “IF fever AND dry cough AND loss of taste, THEN possible COVID-19.”
- **Relationships:** “COVID-19 is a viral disease. Viral diseases are treated with antivirals.”

Without proper knowledge representation, the AI cannot connect these dots.

Common Mistakes Students Make

- **✗** Confusing data with knowledge. Data is raw numbers; knowledge is organized, meaningful information.
- **✗** Thinking one representation fits all. Different problems need different formats.
- **✗** Ignoring the need for inference. Storing facts is useless if the AI cannot reason with them.

Quick Summary

- Knowledge Representation = storing facts, rules, and relationships for AI reasoning
- Types: Declarative (what), Procedural (how), Meta-knowledge (knowledge about knowledge)
- Good representation must be adequate, efficient, and easy to update
- Without knowledge representation, AI cannot reason or make decisions

4.2 Semantic Networks and Frames

Simple Definition

- **Semantic Networks** represent knowledge as a graph of nodes (concepts) connected by edges (relationships).
- **Frames** represent knowledge as structured templates with slots (like filling a form).

Why It Matters

These are two of the most intuitive ways to represent knowledge. They are easy to visualize and understand, making them great for building expert systems and ontologies.

Easy Analogy

- **Semantic Network** = A family tree or a mind map you draw on paper. Circles are people, lines show relationships.
- **Frame** = A admission form at LPU. It has fields (slots) like Name, Course, Address. You fill in the values.

Detailed Explanation

Semantic Networks Structure:

- **Nodes** = Objects, concepts, or events
- **Edges (Links)** = Relationships between nodes
- **Labels on edges** = Type of relationship

Example: Animal Knowledge

```
[Dog] ---is-a---> [Mammal]
|
has-legs          has-legs
|                |
[4]              [warm-blooded]

[Dog] ---eats---> [Meat]
```

Visual Description: Imagine drawing circles on paper. Write “Dog” in one circle, “Mammal” in another. Draw an arrow from Dog to Mammal and label it “is-a.” Draw another arrow from Dog to “Meat” and label it “eats.”

Advantages:

- Easy to understand and visualize
- Flexible — easy to add new nodes and relationships
- Good for inheritance (if Mammal has “warm-blooded,” then Dog inherits it)

Disadvantages:

- Hard to represent complex rules (like “if-then”)
- No standard way to handle uncertainty

- Can become messy for large knowledge bases

Frames Structure:

- A frame is a data structure with **slots** (attributes) and **fillers** (values)
- Each frame represents a class or an instance
- Frames can inherit values from parent frames

Example: Hotel Room Frame

```
Frame: Hotel_Room
  Type: Standard
  Price: 2000
  Has_AC: Yes
  Has_TV: Yes
  View: City

Frame: Hotel_Room_302 (instance of Hotel_Room)
  Type: Standard
  Price: 2000
  Has_AC: Yes
  Has_TV: Yes
  View: Garden <-- overrides parent value
```

Visual Description: Imagine an Excel sheet where each row is a frame and each column is a slot. Some cells are filled automatically from a template (inheritance), and some are filled specifically for that instance.

Advantages:

- Structured and organized
- Supports inheritance (saves repetition)
- Good for representing objects with many attributes

Disadvantages:

- Rigid structure — not all knowledge fits into slots
- Hard to represent procedural knowledge

Practical Example

Example: University Admission System

Semantic Network:

```
[Student] --applies-to--> [LPU]
|
|
|
|
[has-grades]      [CSE Course]
|
|
[85%]             [has-subjects]
|
```

Frame for Student:

```

Frame: Student_Amit
  Name: Amit Kumar
  Age: 20
  Course: CSE
  Grades: 85%
  Backlogs: 0
  Scholarship: Eligible (inherited from Grades > 80% rule)

```

Common Mistakes Students Make

- ❌ Thinking semantic networks and frames are the same. Networks show relationships; frames show attributes.
- ❌ Forgetting that frames support inheritance. A “Car” frame inherits from “Vehicle.”
- ❌ Trying to represent “if-then” rules in semantic networks. Use production rules for that.

Quick Summary

- Semantic Networks = graphs with nodes (concepts) and labeled edges (relationships)
- Frames = structured templates with slots and fillers, support inheritance
- Both are declarative knowledge representations
- Semantic networks are good for relationships; frames are good for object attributes

4.3 Production Systems and Expert Systems**Simple Definition**

- A **Production System** uses “if-then” rules (called productions) to make decisions.
- An **Expert System** is an AI program that mimics a human expert using a knowledge base of rules and an inference engine.

Why It Matters

Expert systems were the first successful commercial AI applications. They are still used in medicine, finance, and engineering where human expertise is valuable but scarce.

Easy Analogy

- **Production System** = A recipe book. “If the curry is too spicy, then add more cream.”
- **Expert System** = A senior doctor who has seen thousands of cases. You tell symptoms, and the doctor gives a diagnosis based on experience (rules).

Detailed Explanation**Production Systems Components:**

1. **Working Memory:** Current facts and data (“The patient has fever”)
2. **Rule Base (Knowledge Base):** Collection of if-then rules

3. **Inference Engine:** Decides which rules to apply and in what order

Example Rules:

```
Rule 1: IF fever AND cough THEN possible_flu
Rule 2: IF possible_flu AND body_ache THEN likely_flu
Rule 3: IF likely_flu AND high_fever THEN prescribe_antiviral
```

How it works:

1. Facts enter working memory (“Patient has fever”)
2. Inference engine checks which rules match
3. Matching rules “fire” (execute)
4. New facts are added to working memory
5. Process repeats until goal is reached or no more rules match

Types of Inference:

- **Forward Chaining:** Start with facts, apply rules, reach conclusion
- Example: Given symptoms, find the disease
- **Backward Chaining:** Start with goal, find rules that lead to it, check if facts support it
- Example: “Is it flu?” → Check if symptoms match flu rules

Expert Systems Architecture:

```
+-----+
| User Interface | <-- Doctor enters symptoms
+-----+
|
+-----+
| Inference Engine | <-- Thinks and reasons
+-----+
|
+-----+
| Knowledge Base | <-- Rules from medical experts
+-----+
```

Key Features:

- **Domain-specific:** Only knows one field (cardiology, tax law, etc.)
- **Explainable:** Can tell you WHY it made a decision (“Because you have fever AND cough”)
- **Consistent:** Never gets tired, never forgets a rule

Famous Examples:

- **MYCIN:** Diagnosed bacterial infections and recommended antibiotics
- **DENDRAL:** Identified chemical compounds from mass spectrometry data
- **R1/XCON:** Configured computer systems for Digital Equipment Corporation

Practical Example

Example: Loan Approval Expert System

Rules:

```
Rule 1: IF salary > 50,000 AND credit_score > 700 THEN eligible_for_loan
Rule 2: IF eligible_for_loan AND loan_amount < 10_lakhs THEN approve
Rule 3: IF eligible_for_loan AND loan_amount > 10_lakhs THEN require_guarantor
```

Scenario:

- Fact: Salary = 60,000, Credit Score = 750, Loan Amount = 5 lakhs
- Forward chaining: Rule 1 fires → eligible_for_loan. Then Rule 2 fires → APPROVE.

Common Mistakes Students Make

- ❌ Confusing forward and backward chaining. Forward = data-driven. Backward = goal-driven.
- ❌ Thinking expert systems can learn. Traditional expert systems do NOT learn — rules are written by humans.
- ❌ Believing expert systems are obsolete. They are still used where explainability and rules matter.

Quick Summary

- Production System = Working Memory + Rule Base + Inference Engine
- Forward Chaining = start with facts, reach conclusion
- Backward Chaining = start with goal, verify with facts
- Expert System = AI that mimics human expert using rules
- Expert systems are explainable, consistent, and domain-specific

4.4 Propositional Logic and First-Order Predicate Logic

Simple Definition

- **Propositional Logic** deals with simple true/false statements (propositions) connected by AND, OR, NOT.
- **First-Order Predicate Logic (FOPL)** is more powerful — it deals with objects, properties, and relationships using variables and quantifiers.

Why It Matters

Logic is the foundation of AI reasoning. It allows AI to draw valid conclusions from known facts. Understanding the difference between propositional and predicate logic helps you know which to use for a given problem.

Easy Analogy

- **Propositional Logic** = Simple true/false quiz questions. “Is Delhi the capital of India? Yes/No”
- **First-Order Logic** = Essay questions where you explain relationships. “All humans are mortal. Socrates is a human. Therefore, Socrates is mortal.” You need subjects, objects, and relationships.

Detailed Explanation

Propositional Logic Basic Elements:

- **Propositions:** Simple statements that are either true or false
- P = “It is raining”
- Q = “I will carry an umbrella”

Logical Connectives:

- **AND ($\wedge$):** $P \wedge Q$ = “It is raining AND I will carry an umbrella” (true only if both are true)
- **OR ($\vee$):** $P \vee Q$ = “It is raining OR I will carry an umbrella” (true if at least one is true)
- **NOT ($\neg$):** $\neg P$ = “It is NOT raining”
- **IMPLIES ($\rightarrow$):** $P \rightarrow Q$ = “If it is raining, then I will carry an umbrella”
- **IFF ($\leftrightarrow$):** $P \leftrightarrow Q$ = “It is raining IF AND ONLY IF I carry an umbrella”

Truth Table for AND:

P	Q	$P \wedge Q$
T	T	T
T	F	F
F	T	F
F	F	F

Limitation: Propositional logic cannot express statements about “all” or “some” objects. You cannot say “All students passed” — you would need a separate proposition for each student.

First-Order Predicate Logic (FOPL) Basic Elements:

- **Constants:** Specific objects (Amit, LPU, Delhi)
- **Variables:** x, y, z (can represent any object)
- **Predicates:** Properties or relations (Passed(x), Loves(x, y), GreaterThan(x, y))
- **Functions:** fatherOf(x), ageOf(x)
- **Quantifiers:**
- **Universal ($\forall$):** “For all” — $\forall x \text{ Student}(x) \rightarrow \text{Human}(x)$ means “All students are humans”
- **Existential ($\exists$):** “There exists” — $\exists x \text{ Student}(x) \wedge \text{Topper}(x)$ means “There exists a student who is a topper”

Example Translation:

- English: “All humans are mortal. Socrates is a human. Therefore, Socrates is mortal.”
- FOPL:
 1. $\forall x \text{ Human}(x) \rightarrow \text{Mortal}(x)$
 2. $\text{Human}(\text{Socrates})$
 3. Therefore: $\text{Mortal}(\text{Socrates})$ [by Modus Ponens]

Inference Rules:

- **Modus Ponens:** If $P \rightarrow Q$ and P is true, then Q is true
- **Modus Tollens:** If $P \rightarrow Q$ and Q is false, then P is false
- **Universal Instantiation:** From $\forall x P(x)$, conclude $P(\text{Amit})$
- **Existential Instantiation:** From $\exists x P(x)$, introduce a new constant c such that $P(c)$

Practical Example

Example 1: Propositional Logic

Given:

- P = “It is hot”
- Q = “The fan is on”
- R = “I am comfortable”

Rules:

- $P \wedge Q \rightarrow R$ (If it is hot AND the fan is on, then I am comfortable)
- P is true
- Q is true

Conclusion: R is true (I am comfortable)

Example 2: First-Order Logic

Given:

- $\forall x \text{Bird}(x) \rightarrow \text{CanFly}(x)$ [All birds can fly — simplified, ignoring penguins!]
- $\text{Bird}(\text{Tweety})$

Conclusion: $\text{CanFly}(\text{Tweety})$

Example 3: Complex FOPL

“Every student who studies hard passes the exam. Amit studies hard. Therefore, Amit passes.”

1. $\forall x (\text{Student}(x) \wedge \text{StudiesHard}(x)) \rightarrow \text{Passes}(x)$
2. $\text{Student}(\text{Amit}) \wedge \text{StudiesHard}(\text{Amit})$
3. Therefore: $\text{Passes}(\text{Amit})$

Common Mistakes Students Make

- ❌ Using propositional logic for problems that need quantifiers (all, some, none).
- ❌ Confusing implication ($\rightarrow$) with biconditional ($\leftrightarrow$). $P \rightarrow Q$ is NOT the same as $P \leftrightarrow Q$.
- ❌ Forgetting that in FOPL, variables need to be bound by quantifiers.
- ❌ Thinking Modus Ponens works backward. From Q , you cannot conclude P .

Quick Summary

- Propositional Logic = simple true/false statements with AND, OR, NOT, IMPLIES
- Cannot express “all” or “some” — limited expressive power

- First-Order Logic = objects, predicates, functions, quantifiers (,)
 - Can express complex relationships and generalizations
 - Inference rules (Modus Ponens, etc.) let AI draw valid conclusions
-

4.5 Reasoning Under Uncertainty: Bayes' Theorem

Simple Definition

Bayes' Theorem is a mathematical formula that updates the probability of a belief when new evidence is observed. It helps AI make decisions when information is uncertain.

Why It Matters

Real-world AI rarely has 100% certain information. A doctor is not 100% sure you have flu — they use symptoms to update their belief. Bayes' Theorem is the mathematical engine behind this reasoning.

Easy Analogy

Imagine you think there is a 10% chance of rain today (your prior belief). Then you see dark clouds (new evidence). Bayes' Theorem tells you how to update your belief — maybe now it is 70% chance of rain. You combined what you already knew with what you just observed.

Detailed Explanation

Basic Probability Concepts

- **P(A)**: Probability of event A happening
- **P(A|B)**: Probability of A given that B has happened (conditional probability)
- **P(A ∩ B)**: Probability of both A and B happening

Bayes' Theorem Formula

$$P(A|B) = \frac{P(B|A) * P(A)}{P(B)}$$

In plain English:

- $P(A|B)$ = “What is the probability of A given B?” (Posterior — updated belief)
- $P(A)$ = “What was the probability of A before seeing B?” (Prior — initial belief)
- $P(B|A)$ = “How likely is the evidence B if A is true?” (Likelihood)
- $P(B)$ = “How likely is the evidence overall?” (Evidence)

Step-by-Step Example Scenario: A medical test for a disease.

Given:

- 1% of people have the disease: $P(\text{Disease}) = 0.01$
- If you have the disease, the test is positive 99% of the time: $P(\text{Positive}|\text{Disease}) = 0.99$

- If you do NOT have the disease, the test is still positive 5% of the time (false positive): $P(\text{Positive} | \text{No Disease}) = 0.05$

Question: If your test is positive, what is the probability you ACTUALLY have the disease?

Step 1: Calculate $P(\text{Positive})$ — total probability of testing positive.

$$\begin{aligned} P(\text{Positive}) &= P(\text{Positive} | \text{Disease}) * P(\text{Disease}) + P(\text{Positive} | \text{No Disease}) * P(\text{No Disease}) \\ &= (0.99 * 0.01) + (0.05 * 0.99) \\ &= 0.0099 + 0.0495 \\ &= 0.0594 \end{aligned}$$

Step 2: Apply Bayes' Theorem.

$$\begin{aligned} P(\text{Disease} | \text{Positive}) &= \frac{P(\text{Positive} | \text{Disease}) * P(\text{Disease})}{P(\text{Positive})} \\ &= (0.99 * 0.01) / 0.0594 \\ &= 0.0099 / 0.0594 \\ &= 0.1667 \text{ or about } 16.67\% \end{aligned}$$

Surprising Result: Even with a positive test, you only have a 16.67% chance of having the disease! This is because the disease is rare (1%) and false positives are common (5%).

Why This Matters for AI

AI systems use Bayes' Theorem for:

- **Medical Diagnosis:** Updating disease probability based on symptoms
- **Spam Filtering:** $P(\text{Spam} | \text{contains "free money"})$ — update spam probability when words are seen
- **Speech Recognition:** $P(\text{word} | \text{sound heard})$ — figure out what word was spoken
- **Autonomous Vehicles:** $P(\text{pedestrian} | \text{sensor reading})$ — decide if that blob is a person

Naive Bayes Classifier

A popular AI algorithm based on Bayes' Theorem. It assumes all features are independent (which is often wrong but works surprisingly well).

Example: Spam Detection

$$P(\text{Spam} | \text{"free", "money", "click"}) = P(\text{"free"} | \text{Spam}) * P(\text{"money"} | \text{Spam}) * P(\text{"click"} | \text{Spam}) * P(\text{Spam}) / P(\text{evidence})$$

Common Mistakes Students Make

- **✗** Forgetting that $P(B)$ in the denominator is the TOTAL probability, not just $P(B|A)$.
- **✗** Thinking a positive test means you definitely have the disease. Base rate (prior probability) matters hugely.
- **✗** Confusing $P(A|B)$ with $P(B|A)$. These are NOT the same.
- **✗** Ignoring false positives and false negatives in medical test scenarios.

Quick Summary

- Bayes' Theorem: $P(A|B) = P(B|A) * P(A) / P(B)$
 - Updates beliefs when new evidence arrives
 - Prior + Likelihood + Evidence = Posterior
 - Counterintuitive result: Even accurate tests can give misleading results for rare diseases
 - Used in spam filters, medical diagnosis, speech recognition, and more
-

UNIT IV: KEY POINTS TO REMEMBER

- Knowledge Representation stores facts, rules, and relationships for AI reasoning
 - Semantic Networks = nodes (concepts) + labeled edges (relationships), good for inheritance
 - Frames = structured templates with slots and fillers, support inheritance from parent frames
 - Production Systems = Working Memory + Rule Base + Inference Engine
 - Forward Chaining = data-driven (facts $\rightarrow$ conclusion); Backward Chaining = goal-driven (goal $\rightarrow$ verify facts)
 - Expert Systems mimic human experts using rules; they are explainable and domain-specific
 - Propositional Logic = simple true/false with AND, OR, NOT, IMPLIES; cannot express “all” or “some”
 - First-Order Predicate Logic = objects, predicates, functions, quantifiers (,); much more expressive
 - Modus Ponens: If $P \rightarrow Q$ and P , then Q
 - Bayes' Theorem updates probabilities: $P(A|B) = P(B|A) * P(A) / P(B)$
 - Even with positive test results, prior probability (base rate) dramatically affects the conclusion
-

UNIT IV: SELF-CHECK QUESTIONS

1. What are the three components of a production system?
 2. How does a semantic network represent knowledge? Draw a simple example.
 3. What is the difference between forward chaining and backward chaining?
 4. Why is propositional logic not enough for expressing “All birds can fly”?
 5. Write “All students are hard-working” in First-Order Predicate Logic.
 6. In Bayes' Theorem, what do $P(A)$, $P(B|A)$, and $P(A|B)$ represent?
 7. What is the advantage of frames over semantic networks?
 8. Why are expert systems called “explainable AI”?
-

UNIT IV: SCENARIO-BASED MCQs

Question 1

A hospital builds an AI system that stores medical knowledge as “if-then” rules. For example: “IF fever AND dry cough AND loss of taste THEN possible COVID.” The system also stores current patient symptoms separately and applies matching rules to reach a diagnosis. What type of system is this?

- A) Semantic Network

- B) Production System
- C) Frame-Based System
- D) Neural Network

Correct Answer: B

Explanation: This is a classic Production System — it has a Rule Base (if-then rules), Working Memory (patient symptoms), and an Inference Engine (applies rules). Semantic networks (A) use nodes and edges, not if-then rules. Frames (C) use slots and fillers. Neural networks (D) learn from data, not from explicit rules.

Question 2

A knowledge engineer is designing an AI to represent relationships in a family tree. She draws circles for each person and arrows labeled “is-father-of,” “is-sister-of,” etc. Which knowledge representation technique is she using?

- A) Frames
- B) Production Rules
- C) Semantic Networks
- D) Propositional Logic

Correct Answer: C

Explanation: Circles (nodes) connected by labeled arrows (edges) is the definition of a Semantic Network. Frames (A) use structured templates. Production rules (B) use if-then statements. Propositional logic (D) uses true/false statements without objects or relationships.

Question 3

An expert system for loan approval starts with the question: “Should we approve this loan?” It then checks rules backward to see if the applicant’s data supports approval. For example, it checks “Is the credit score > 700?” and “Is the salary sufficient?” This reasoning method is called:

- A) Forward Chaining
- B) Backward Chaining
- C) Breadth-First Reasoning
- D) Bayesian Reasoning

Correct Answer: B

Explanation: Starting with the goal (“Should we approve?”) and working backward to verify supporting facts is Backward Chaining. Forward chaining (A) would start with the applicant’s data and derive the conclusion. Breadth-first (C) is a search algorithm, not a reasoning method. Bayesian reasoning (D) uses probabilities, not rule chaining.

Question 4

A student writes: “ $P \rightarrow Q$ means Q implies P.” Is the student correct?

- A) Yes, implication is commutative
- B) No, $P \rightarrow Q$ means “If P then Q,” not “If Q then P”
- C) Yes, in logic, $P \rightarrow Q$ and $Q \rightarrow P$ are equivalent
- D) No, $P \rightarrow Q$ means P and Q are both true

Correct Answer: B

Explanation: $P \rightarrow Q$ means “If P is true, then Q is true.” It does NOT mean “If Q is true, then P is true.” Implication is NOT commutative. Option A and C are wrong. Option D is wrong because $P \rightarrow Q$ is true even when P is false (a false premise makes the implication vacuously true).

Question 5

A medical test for a rare disease has 99% accuracy. Only 0.5% of the population has the disease. If a random person tests positive, which statement is MOST likely true?

- A) They almost certainly have the disease
- B) They probably do NOT have the disease because false positives outnumber true positives
- C) The test is useless because the disease is rare
- D) We need more information about the test’s sensitivity

Correct Answer: B

Explanation: This is the classic Bayes’ Theorem trap. In a large population, even a 1% false positive rate produces MANY more false positives than true positives (because the disease is rare). So most positive tests are actually false positives. Option A is the common intuitive mistake. Option C is wrong — the test is still useful but needs confirmation. Option D is wrong because the question gives enough information to reason qualitatively.

Question 6

Which of the following CANNOT be expressed in Propositional Logic?

- A) “It is raining AND the ground is wet”
- B) “All humans are mortal”
- C) “The light is on OR the light is off”
- D) “If it rains, then the ground is wet”

Correct Answer: B

Explanation: “All humans are mortal” requires a quantifier () and a predicate (Human, Mortal). Propositional logic has no quantifiers or predicates — only simple propositions. Options A, C, and D can all be written with AND, OR, and IMPLIES in propositional logic.

Question 7

In First-Order Predicate Logic, the statement $\forall x (\text{Student}(x) \wedge \text{Hardworking}(x)) \rightarrow \text{Success}(x)$ means:

- A) There exists a student who is hardworking and successful
- B) For all x, if x is a student and x is hardworking, then x is successful
- C) All students are hardworking, and all hardworking people are successful
- D) Some students are successful because they are hardworking

Correct Answer: B

Explanation: $\forall x$ means “for all x.” The implication $\rightarrow$ means “if...then.” So it reads: “For all x, if x is a student AND x is hardworking, THEN x is successful.” Option A uses existential (), not universal. Option C changes the meaning by separating the statements. Option D also implies existence, not universality.

Question 8

A frame for “Car” has slots: Make, Model, Year, Color. A frame for “ElectricCar” inherits from “Car” and adds slots: BatteryCapacity, Range. When an instance “Tesla_Model_3” is created, it inherits Make, Model, Year, Color from Car and gets BatteryCapacity and Range from ElectricCar. This demonstrates:

- A) Forward Chaining
- B) Frame Inheritance
- C) Semantic Network Traversal
- D) Bayesian Updating

Correct Answer: B

Explanation: This is Frame Inheritance — child frames (ElectricCar) inherit slots from parent frames (Car). Forward chaining (A) is rule-based reasoning. Semantic network traversal (C) involves following edges in a graph. Bayesian updating (D) involves probability revision.

Question 9

A spam filter uses the words in an email to calculate $P(\text{Spam} \mid \text{words})$. It assumes each word independently contributes to the spam probability. This AI technique is based on:

- A) Propositional Logic
- B) First-Order Logic
- C) Bayes' Theorem
- D) Semantic Networks

Correct Answer: C

Explanation: $P(\text{Spam} \mid \text{words})$ is a conditional probability calculated using Bayes' Theorem. The Naive Bayes classifier specifically does this. Propositional logic (A) and first-order logic (B) are symbolic reasoning, not probabilistic. Semantic networks (D) represent relationships, not probabilities.

Question 10

An AI system needs to reason: “If a person has a high fever AND a rash AND the rash appeared after the fever, THEN the person likely has measles.” The AND conditions and the temporal relationship (appeared after) are best represented using:

- A) Simple propositions only
- B) Production rules with compound conditions
- C) A single semantic network node
- D) A frame with only one slot

Correct Answer: B

Explanation: Production rules naturally handle compound conditions (IF A AND B AND C THEN D). Simple propositions (A) cannot capture the relationship between symptoms. A single node (C) or single slot (D) cannot represent this complex rule.

Question 11

In an expert system, the component that decides which rule to apply next when multiple rules match the current facts is called the:

- A) Knowledge Base
- B) Working Memory
- C) Inference Engine
- D) User Interface

Correct Answer: C

Explanation: The Inference Engine is the “brain” that matches rules with facts and decides the firing order. The Knowledge Base (A) stores rules. Working Memory (B) stores current facts. The User Interface (D) interacts with humans.

Question 12

A diagnostic AI is given: $P(\text{Disease}) = 0.02$, $P(\text{Symptom}|\text{Disease}) = 0.9$, $P(\text{Symptom}|\text{No Disease}) = 0.1$. A patient shows the symptom. Using Bayes' Theorem, what happens to $P(\text{Disease}|\text{Symptom})$ compared to $P(\text{Disease})$?

- A) It decreases because the symptom is common even without the disease
- B) It increases because the symptom is more likely with the disease
- C) It stays the same because one symptom is not enough
- D) It becomes exactly 0.9

Correct Answer: B

Explanation: Since $P(\text{Symptom}|\text{Disease}) = 0.9$ is much higher than $P(\text{Symptom}|\text{No Disease}) = 0.1$, seeing the symptom INCREASES the probability of the disease. Let's verify: $P(\text{Symptom}) = 0.9 \cdot 0.02 + 0.1 \cdot 0.98 = 0.018 + 0.098 = 0.116$. $P(\text{Disease}|\text{Symptom}) = (0.9 * 0.02) / 0.116 = 0.018 / 0.116 \approx 0.155$. This is higher than the prior 0.02. Option A is wrong — while the symptom is somewhat common without disease, it is MUCH more common WITH disease. Option C is wrong — probability does change. Option D is wrong — 0.9 is the likelihood, not the posterior.

UNIT V: INTELLIGENT AGENTS

5.1 Introduction to Intelligent Agents and Agent Architecture

Simple Definition

An **intelligent agent** is anything that can perceive its environment through sensors and act upon that environment through actuators to achieve goals. **Agent architecture** is the design of how the agent's

brain processes perceptions and decides actions.

Why It Matters

Intelligent agents are the fundamental building blocks of AI systems. Understanding agents helps you design AI that interacts with the real world — from chatbots to self-driving cars to robots.

Easy Analogy

Think of an intelligent agent like a security guard at a college gate:

- **Sensors:** His eyes and ears (sees students, hears sounds)
- **Brain:** His mind (decides if someone should be allowed in)
- **Actuators:** His hands (opens gate, stops someone, calls for help)
- **Environment:** The college campus and everyone in it

Detailed Explanation

What is an Agent? An agent is defined by its interaction with the environment:

```
Environment → [Sensors] → Agent Brain → [Actuators] → Environment
```

Key Components:

1. **Sensors:** How the agent perceives (camera, microphone, keyboard, temperature sensor)
2. **Actuators:** How the agent acts (motor, speaker, screen display, robot arm)
3. **Agent Function:** The mapping from perceptions to actions (the “brain”)
4. **Agent Program:** The actual code that implements the agent function

Example Agents:

- **Thermostat:** Sensor = temperature sensor; Actuator = heater/AC switch
- **Roomba (Vacuum Robot):** Sensor = camera, bump sensor; Actuator = wheels, vacuum motor
- **Chatbot:** Sensor = text input; Actuator = text output
- **Self-driving car:** Sensors = cameras, LiDAR, GPS; Actuators = steering, brakes, accelerator

Agent Architecture Architecture is HOW the agent is built internally. Common architectures:

1. **Simple Reflex:** If perception matches a condition, do action. No memory.
2. **Model-Based:** Maintains an internal model of the world. Has memory.
3. **Goal-Based:** Has goals and chooses actions that lead to goals.
4. **Utility-Based:** Has preferences (utility) and chooses actions that maximize happiness/satisfaction.
5. **Learning-Based:** Improves over time from experience.

Practical Example

Example: Smart AC Agent

- **Sensors:** Temperature sensor, humidity sensor, occupancy sensor

- **Brain:** If temperature $> 26^{\circ}\text{C}$ AND room is occupied, turn on AC. If room is empty for 30 minutes, turn off AC.
- **Actuators:** AC compressor, fan speed controller
- **Environment:** The room and its occupants

Common Mistakes Students Make

- **✗** Confusing “agent” with “robot.” A chatbot is an agent too — it perceives text and acts with text.
- **✗** Thinking all agents are intelligent. A simple thermostat is an agent but not very intelligent.
- **✗** Forgetting that the environment shapes the agent’s design. An agent for a static environment is very different from one for a dynamic environment.

Quick Summary

- Agent = Perceives environment through sensors + Acts through actuators
 - Agent function = mapping from perceptions to actions
 - Agent architecture = how the brain is designed (reflex, model-based, goal-based, etc.)
 - Examples range from simple thermostats to self-driving cars
-

5.2 PEAS Framework

Simple Definition

PEAS is a framework to describe an agent by defining its **P**erformance measure, **E**nvironment, **A**ctuators, and **S**ensors.

Why It Matters

PEAS helps you systematically design any intelligent agent. Before building an AI, you should always ask: What does success look like? Where does it operate? What can it do? What can it see?

Easy Analogy

Before you buy a phone, you think about:

- **Performance:** How fast should it be? Good camera? Long battery?
- **Environment:** Will you use it in dusty areas? Underwater? In bright sunlight?
- **Actuators:** What can it do? Call, text, take photos, run apps?
- **Sensors:** What can it detect? Touch, light, GPS, fingerprint?

PEAS does the same for AI agents.

Detailed Explanation

P — Performance Measure How do we measure if the agent is doing a good job?

- Should be objective and measurable
- Example for taxi agent: Safety, speed, comfort, legality, profit

E — Environment Where does the agent operate? What are the characteristics?

Environment Properties:

- **Fully Observable vs. Partially Observable:** Can the agent see everything? (Chess = fully observable; Poker = partially observable)
- **Deterministic vs. Stochastic:** Are outcomes predictable? (Chess = deterministic; Dice game = stochastic)
- **Episodic vs. Sequential:** Does each decision stand alone? (Quality inspection = episodic; Chess = sequential)
- **Static vs. Dynamic:** Does the environment change while the agent thinks? (Crossword = static; Driving = dynamic)
- **Discrete vs. Continuous:** Are there finite or infinite states/actions? (Chess = discrete; Driving = continuous)
- **Single-agent vs. Multi-agent:** Is the agent alone or with others?

A — Actuators What actions can the agent take?

- Example for taxi: Steering wheel, accelerator, brake, horn, indicator

S — Sensors What can the agent perceive?

- Example for taxi: Camera, speedometer, GPS, accelerometer, microphone

Practical Example

PEAS for a Self-Driving Car:

PEAS	Description
Performance	Reach destination safely, follow traffic rules, minimize travel time, maximize passenger comfort
Environment	Roads, traffic, pedestrians, weather, other vehicles, traffic signals
Actuators	Steering, brakes, accelerator, horn, headlights, indicators
Sensors	Cameras, LiDAR, radar, GPS, speedometer, accelerometer

PEAS for a Medical Diagnosis Chatbot:

PEAS	Description
Performance	Correct diagnosis, minimize false alarms, user-friendly interaction
Environment	Patient symptoms, medical history, user queries
Actuators	Text responses, referral suggestions, alert to human doctor
Sensors	Text input, symptom descriptions, patient history database

Common Mistakes Students Make

- **✗** Defining vague performance measures. “Be good” is not measurable. “Reduce travel time by 20%” is.

- ❌ Ignoring environment properties. A static environment agent will fail in a dynamic one.
- ❌ Confusing actuators with sensors. Actuators DO things; sensors DETECT things.

Quick Summary

- PEAS = Performance, Environment, Actuators, Sensors
 - Always define PEAS before designing an agent
 - Environment properties (observable, deterministic, static, etc.) determine agent design
 - Good performance measures are specific and measurable
-

5.3 Rational Agents and Agent Environments

Simple Definition

A **rational agent** is one that does the best possible thing given what it knows. It maximizes its performance measure. The **agent environment** is the world the agent operates in.

Why It Matters

Rationality is the GOAL of AI. We do not just want agents that work; we want agents that make the BEST decisions possible with available information.

Easy Analogy

Imagine two students preparing for an exam:

- **Student A** studies random chapters. Might pass, might fail.
- **Student B** analyzes past papers, focuses on high-weightage topics, and practices MCQs. Student B is being RATIONAL — making the best decision with available information.

A rational agent is like Student B.

Detailed Explanation

What is Rationality? A rational agent selects actions that are expected to maximize its performance measure, given:

1. The percept sequence (everything it has perceived so far)
2. Its built-in knowledge
3. Its ability to reason

Important: Rationality is NOT the same as perfection. A rational agent does the BEST it can with what it KNOWS. If information is missing, it might make a “rational” decision that turns out wrong.

Example: A doctor who correctly diagnoses 95% of cases is rational, even if they occasionally miss a rare disease they have never seen before.

Types of Environments

Property	Description	Example
Fully Observable	Agent can see complete state	Chess
Partially Observable	Agent sees only part of state	Poker, driving with fog
Deterministic	Next state is fully determined by current state and action	Chess
Stochastic	Next state has randomness	Dice games, stock market
Episodic	Each episode is independent	Quality inspection on assembly line
Sequential	Current action affects future	Chess, driving
Static	Environment does not change while agent thinks	Crossword puzzle
Dynamic	Environment changes while agent thinks	Driving, trading stocks
Discrete	Finite states and actions	Chess
Continuous	Infinite states and actions	Driving, robot arm movement
Single-agent	Only one agent	Crossword
Multi-agent	Multiple agents interact	Chess, football, traffic

The Hardest Environment: Partially observable, stochastic, sequential, dynamic, continuous, multi-agent. Example: Driving in Indian city traffic during monsoon!

Practical Example

Example: Rational Taxi Driver Agent

Scenario: A taxi must pick up a passenger from the airport and drop them at a hotel.

Rational behavior:

- Check GPS for traffic (uses perception)
- Choose the fastest route (maximizes performance: minimize time)
- Follow traffic rules (part of performance measure: safety and legality)
- Adjust AC based on passenger comfort (uses sensors)

Irrational behavior:

- Taking a longer route because “it looks nice”
- Ignoring traffic signals
- Not checking if passenger is in the car before driving

Common Mistakes Students Make

- **✗** Confusing rationality with omniscience. A rational agent does not know everything — it just makes the best decision with available info.
- **✗** Thinking rational agents never fail. They can fail if the environment is unpredictable or information is wrong.
- **✗** Forgetting that rationality depends on the performance measure. A “rational” military drone and a “rational” hospital robot have very different behaviors.

Quick Summary

- Rational agent = maximizes performance measure given what it knows
 - Rationality = perfection; rationality = best possible with available information
 - Environment properties determine how complex the agent must be
 - Hardest environments: partially observable, stochastic, sequential, dynamic, continuous, multi-agent
-

5.4 Types of Agents

Simple Definition

Agents can be classified by how they make decisions: simple reflex, model-based, goal-based, utility-based, and learning agents.

Why It Matters

Each agent type solves different problems. A thermostat needs only reflex, but a self-driving car needs learning. Knowing the types helps you design the right agent for the job.

Easy Analogy

Think of these agents as levels of student maturity:

- **Simple Reflex** = A baby. Sees fire, pulls hand away. No thinking.
- **Model-Based** = A school kid. Remembers that touching fire hurts. Uses memory.
- **Goal-Based** = A college student. Has a goal (get a job) and plans steps to achieve it.
- **Utility-Based** = A smart adult. Weighs options — this job pays more but that job has better growth.
- **Learning** = A wise elder. Learns from every experience and gets better over time.

Detailed Explanation

1. Simple Reflex Agent How it works:

- Uses condition-action rules: “If PERCEPT, then ACTION”
- No memory of past states
- Acts ONLY on current perception

Example:

```
IF temperature > 30°C THEN turn_on_AC  
IF obstacle_detected THEN stop
```

Visual Description: Like a knee-jerk reflex. The doctor hits your knee, your leg kicks. No brain processing needed.

Advantages: Simple, fast, reliable for simple tasks **Disadvantages:** Cannot handle partially observable environments, no memory, no planning

2. Model-Based Reflex Agent How it works:

- Has an **internal model** of the world (memory)
- Updates the model based on perceptions
- Uses “If PERCEPT and STATE, then ACTION”

Example: A robot vacuum knows the room layout (model). When it hits a wall, it updates its position in the model and turns. It remembers where it has already cleaned.

Visual Description: Like driving with a map. You do not just look at the road; you also remember the map in your head.

Advantages: Can handle partially observable environments, more flexible **Disadvantages:** Still does not plan for the future, only reacts

3. Goal-Based Agent How it works:

- Has explicit **goals**
- Considers: “Will this action bring me closer to my goal?”
- Can plan sequences of actions

Example: A delivery robot’s goal is “Deliver package to Room 302.” It plans: go to elevator → go to 3rd floor → find Room 302 → deliver.

Visual Description: Like a student with a study plan. Every action is evaluated against the goal of passing the exam.

Advantages: Can plan, flexible, handles complex tasks **Disadvantages:** Needs clear goals, does not handle conflicting goals well

4. Utility-Based Agent How it works:

- Has a **utility function** that measures “happiness” or “satisfaction”
- Chooses the action that MAXIMIZES expected utility
- Can handle trade-offs between multiple goals

Example: A self-driving car must balance speed vs. safety vs. comfort. A utility function assigns scores:

- Speed: +10 points per minute saved
- Safety: -1000 points per accident risk
- Comfort: +5 points per smooth ride

The agent chooses the action with the highest total utility.

Visual Description: Like choosing a college. LPU might score high on placement (utility), while another college scores high on campus life. You weigh all factors.

Advantages: Handles trade-offs, optimizes overall satisfaction **Disadvantages:** Hard to design good utility functions, computationally expensive

5. Learning Agent How it works:

- Has a **learning element** that improves performance from experience
- Includes all previous types + ability to adapt
- Four components: Learning element, Performance element, Critic, Problem generator

Example: AlphaGo started knowing the rules of Go but got better by playing millions of games against itself. It learned strategies no human had discovered.

Visual Description: Like a student who analyzes every test they take, learns from mistakes, and improves their study strategy.

Advantages: Adapts to new situations, improves over time **Disadvantages:** Needs lots of data, can learn bad behaviors, hard to debug

Comparison Table

Agent Type	Memory	Goals	Utility	Learning	Example
Simple Reflex	No	No	No	No	Thermostat
Model-Based	Yes	No	No	No	Robot vacuum with map
Goal-Based	Yes	Yes	No	No	Pathfinding robot
Utility-Based	Yes	Yes	Yes	No	Self-driving car
Learning	Yes	Yes	Yes	Yes	AlphaGo, ChatGPT

Practical Example

Scenario: Smart Home System

- **Simple Reflex:** Light turns on when motion is detected
- **Model-Based:** Light turns on when motion is detected AND it is dark (remembers time of day)
- **Goal-Based:** System aims to “maintain 24°C comfort” — turns AC/heater on/off as needed
- **Utility-Based:** System balances comfort (24°C) vs. electricity bill (lower is better) vs. noise (quiet at night)
- **Learning:** System learns your schedule — pre-cools the room before you usually arrive home

Common Mistakes Students Make

- **✗** Thinking all agents should be learning agents. Sometimes a simple reflex agent is cheaper and more reliable.
- **✗** Confusing goal-based with utility-based. Goal-based asks “Does this reach the goal?” Utility-based asks “How happy does this make me?”
- **✗** Forgetting that model-based agents need memory. Simple reflex agents do NOT have memory.

Quick Summary

- Simple Reflex = condition-action rules, no memory

- Model-Based = adds internal world model (memory)
 - Goal-Based = adds goals and planning
 - Utility-Based = adds preferences and trade-offs
 - Learning = adds ability to improve from experience
 - Complexity increases: Reflex < Model < Goal < Utility < Learning
-

5.5 Autonomous Systems and Multi-Agent Systems

Simple Definition

- **Autonomous systems** operate without human intervention, making their own decisions.
- **Multi-agent systems (MAS)** involve multiple agents interacting, cooperating, or competing to achieve individual or shared goals.

Why It Matters

Most real-world problems involve multiple entities working together (or against each other). Traffic, supply chains, disaster response — all need multi-agent coordination.

Easy Analogy

- **Autonomous System** = A student who studies alone, manages their own schedule, and does not need parents to remind them.
- **Multi-Agent System** = A group project where each student has a role, but you must coordinate to submit on time. Some cooperate; some might freeload!

Detailed Explanation

Autonomous Systems An autonomous system:

- Senses its environment
- Makes decisions without human input
- Acts on those decisions
- Learns and adapts over time

Levels of Autonomy:

1. **Human-operated:** Human makes all decisions (traditional remote-controlled car)
2. **Human-delegated:** Human sets goal, machine executes (cruise control)
3. **Human-supervised:** Machine acts, human monitors and can override (Tesla Autopilot)
4. **Fully autonomous:** Machine handles everything, human is passenger (future goal)

Challenges:

- Safety: What if the system makes a wrong decision?
- Ethics: Who is responsible if an autonomous car hits someone?
- Trust: Will humans trust machines with critical decisions?

Multi-Agent Systems (MAS) Key Concepts:

- **Agent:** An autonomous entity with its own goals and behaviors
- **Environment:** The shared world where agents operate
- **Interaction:** Agents communicate, cooperate, or compete

Types of MAS:

1. **Cooperative MAS:** Agents work together toward a common goal
 - Example: Warehouse robots that divide tasks and avoid collisions
2. **Competitive MAS:** Agents have conflicting goals
 - Example: Chess-playing AI vs. human (or two trading algorithms competing)
3. **Mixed MAS:** Some cooperation, some competition
 - Example: Ride-sharing drivers (cooperate on platform, compete for passengers)

Communication in MAS:

- Agents share information through messages
- They negotiate (“You take Package A, I take Package B”)
- They coordinate to avoid conflicts

Coordination Mechanisms:

- **Centralized:** One master agent decides for all (like a team leader)
- **Decentralized:** Agents make local decisions but follow shared rules (like traffic rules)
- **Distributed:** No central control; agents self-organize (like a flock of birds)

Practical Example

Example 1: Warehouse Robots (Cooperative MAS)

Amazon warehouses use hundreds of robots that:

- Communicate their positions
- Divide shelf-fetching tasks
- Avoid collisions by negotiating paths
- Cooperate to deliver items to packing stations

Example 2: Autonomous Traffic Management (Mixed MAS)

Self-driving cars in a city:

- Cooperate: Share traffic data to help everyone avoid jams
- Compete: Each wants the fastest route for its passenger
- Coordinate: Follow traffic rules and negotiate right-of-way at intersections

Common Mistakes Students Make

- **✗** Thinking autonomous means “no human involvement at all.” Most systems are supervised autonomy.
- **✗** Confusing multi-agent with multi-tasking. A single agent doing many tasks is NOT multi-agent.

- ❌ Assuming all multi-agent systems cooperate. Competition and conflict are common.

Quick Summary

- Autonomous systems = operate independently without human control
- Levels: Human-operated → Delegated → Supervised → Fully autonomous
- Multi-agent systems = multiple agents interacting in a shared environment
- MAS can be cooperative, competitive, or mixed
- Coordination: Centralized, Decentralized, or Distributed

5.6 Human-AI Collaboration and Applications of Intelligent Agents

Simple Definition

Human-AI collaboration means humans and AI working together, each contributing their strengths. Intelligent agents are used in countless real-world applications.

Why It Matters

The future is not “AI replaces humans” — it is “AI + humans together.” Understanding collaboration helps you design AI that augments human capabilities rather than replacing them.

Easy Analogy

Think of a pilot and autopilot in an airplane:

- **Autopilot (AI):** Handles routine flying, maintains altitude, follows GPS route
- **Pilot (Human):** Makes critical decisions, handles emergencies, communicates with air traffic control
- **Together:** Safer and more efficient than either alone

Detailed Explanation

Human-AI Collaboration Models

1. **AI as Assistant:** AI suggests, human decides
 - Example: Grammarly suggests corrections; you decide whether to accept them.
2. **AI as Augmentation:** AI handles routine tasks; human focuses on complex ones
 - Example: AI reads X-rays and highlights suspicious areas; radiologist makes final diagnosis.
3. **Human-in-the-Loop:** AI operates autonomously but human can intervene
 - Example: Drone delivers package but human can take control if something goes wrong.
4. **Human-on-the-Loop:** AI operates autonomously; human monitors and can override
 - Example: Autonomous car drives itself; human driver ready to take the wheel.

Why Collaboration is Better Than Replacement

Human Strengths	AI Strengths
Creativity and intuition	Processing speed and scale
Ethical judgment	Consistency and tirelessness

Table 14 – continued

Human Strengths	AI Strengths
Handling novel situations	Pattern recognition in big data
Emotional intelligence	24/7 operation without fatigue
Common sense reasoning	Precision in calculations

Together: Humans provide judgment and creativity; AI provides speed and data analysis.

Applications of Intelligent Agents

1. **Personal Assistants:** Siri, Alexa, Google Assistant — help with reminders, searches, smart home control
2. **Healthcare:** Diagnostic agents, drug discovery agents, surgical robots
3. **Finance:** Trading agents, fraud detection agents, customer service chatbots
4. **Transportation:** Self-driving cars, traffic management agents, delivery drones
5. **Manufacturing:** Predictive maintenance agents, quality inspection robots
6. **Education:** Personalized tutoring agents, automated grading systems
7. **Entertainment:** Game AI, content recommendation agents (Netflix, Spotify)
8. **Security:** Surveillance agents, intrusion detection systems

Practical Example

Example: AI in Surgery (Da Vinci Robot)

- **AI/Agent Role:** Stabilizes instruments, filters hand tremors, provides 3D visualization
- **Human Role:** Surgeon makes all decisions, controls the robot's movements
- **Collaboration:** The robot makes the surgeon's hands more precise; the surgeon provides the expertise and judgment
- **Result:** Smaller incisions, faster recovery, fewer complications

Common Mistakes Students Make

- **✗** Believing AI will fully replace doctors, teachers, and drivers. In reality, AI augments these roles.
- **✗** Ignoring the ethical responsibility in human-AI collaboration. When AI is wrong, the human is still accountable.
- **✗** Thinking collaboration is easy. Designing AI that humans can trust and understand is very difficult.

Quick Summary

- Human-AI collaboration = combining human judgment with AI speed and consistency
- Models: AI as Assistant, Augmentation, Human-in-the-Loop, Human-on-the-Loop
- Humans bring creativity, ethics, and common sense; AI brings speed, scale, and precision
- Intelligent agents are used in virtually every industry today

UNIT V: KEY POINTS TO REMEMBER

- Agent = perceives environment (sensors) + acts on environment (actuators)

- PEAS = Performance measure, Environment, Actuators, Sensors
 - Environment properties: Observable, Deterministic, Episodic/Sequential, Static/Dynamic, Discrete/Continuous, Single/Multi-agent
 - Rational agent = maximizes performance measure given what it knows (not omniscient)
 - Agent types by complexity: Simple Reflex → Model-Based → Goal-Based → Utility-Based → Learning
 - Simple Reflex: condition-action rules, no memory
 - Model-Based: adds internal world model
 - Goal-Based: adds planning toward goals
 - Utility-Based: adds preferences and trade-offs
 - Learning: improves from experience
 - Autonomous systems operate independently; levels range from human-operated to fully autonomous
 - Multi-agent systems: cooperative, competitive, or mixed
 - Coordination: Centralized, Decentralized, Distributed
 - Human-AI collaboration is the future: AI assists, augments, and works with humans
-

UNIT V: SELF-CHECK QUESTIONS

1. Define an intelligent agent and list its four components.
 2. What does PEAS stand for? Give a PEAS description for a self-driving car.
 3. What is the difference between a fully observable and partially observable environment?
 4. Why is a rational agent NOT the same as an omniscient agent?
 5. List the five types of agents in order of increasing complexity.
 6. What is the difference between a goal-based agent and a utility-based agent?
 7. Give two examples of cooperative multi-agent systems.
 8. Why is human-AI collaboration better than full AI replacement in healthcare?
-

UNIT V: SCENARIO-BASED MCQs

Question 1

A team is designing an AI for a vacuum cleaning robot. The robot has bump sensors, a camera, and a dirt sensor. It can move forward, backward, turn, and activate suction. The goal is to clean the entire room while avoiding obstacles and returning to its charging station when battery is low. Using the PEAS framework, what is the PERFORMANCE measure for this agent?

- A) The camera resolution and dirt sensor accuracy
- B) Percentage of floor cleaned, time taken, battery efficiency, obstacle avoidance
- C) The speed of the wheels and suction power
- D) The size of the room and number of obstacles

Correct Answer: B

Explanation: Performance measures evaluate how WELL the agent does its job — percentage cleaned, time, efficiency, and safety. Option A describes sensors. Option C describes actuators. Option D describes the environment, not performance.

Question 2

A chess-playing AI can see the entire board, knows all rules, and the outcome of every move is predictable. Which environment properties does chess have?

- A) Partially observable, stochastic, sequential
- B) Fully observable, deterministic, sequential, discrete, single-agent
- C) Fully observable, deterministic, sequential, discrete, multi-agent
- D) Partially observable, deterministic, episodic

Correct Answer: C

Explanation: Chess is fully observable (see whole board), deterministic (no dice rolls), sequential (current move affects future), discrete (finite moves), and multi-agent (two players). Option B is wrong because chess has two agents (players). Option A and D are wrong because chess is fully observable and deterministic.

Question 3

A thermostat turns on the AC when the temperature rises above 26°C. It does not remember past temperatures or plan for future weather. What type of agent is this?

- A) Model-Based Reflex Agent
- B) Goal-Based Agent
- C) Simple Reflex Agent
- D) Learning Agent

Correct Answer: C

Explanation: The thermostat uses a simple condition-action rule (“If temp > 26, turn on AC”) with no memory or planning. This is the definition of a Simple Reflex Agent. Model-based (A) would remember past states. Goal-based (B) would plan. Learning (D) would adapt over time.

Question 4

A self-driving car must choose between two routes: Route A is 5 minutes faster but has a 1% chance of an accident. Route B is slower but perfectly safe. The car evaluates both

options and chooses Route B because safety is weighted much higher than speed in its decision function. What type of agent is this demonstrating?

- A) Simple Reflex Agent
- B) Goal-Based Agent
- C) Utility-Based Agent
- D) Model-Based Agent

Correct Answer: C

Explanation: The agent is making a TRADE-OFF between speed and safety using a utility function that assigns different weights to different outcomes. This is the hallmark of a Utility-Based Agent. A goal-based agent (B) would simply try to reach the destination without weighing trade-offs. Simple reflex (A) and model-based (D) do not handle such complex preferences.

Question 5

In a warehouse, 50 robots work together to fetch items from shelves and deliver them to packing stations. They communicate their locations, divide tasks, and avoid collisions. This is an example of:

- A) A single learning agent with 50 actuators
- B) A cooperative multi-agent system
- C) A competitive multi-agent system
- D) A centralized autonomous system

Correct Answer: B

Explanation: Multiple robots working together toward a shared goal (fulfilling orders) with communication and coordination is a cooperative multi-agent system. Option A is wrong because these are 50 separate agents, not one agent. Option C is wrong because they cooperate, not compete. Option D is wrong because warehouse robots are typically decentralized, not centrally controlled.

Question 6

A medical AI analyzes X-rays and highlights suspicious areas in red for the radiologist to review. The radiologist makes the final diagnosis. This is an example of:

- A) Fully autonomous AI
- B) Human-AI collaboration with AI as augmentation

- C) Simple reflex agent
- D) Competitive multi-agent system

Correct Answer: B

Explanation: The AI augments the radiologist's ability by preprocessing and highlighting, but the human makes the final decision. This is "AI as Augmentation." Option A is wrong because it is not fully autonomous. Option C is wrong because medical diagnosis is far too complex for simple reflex. Option D is wrong because there is no competition.

Question 7

An environment where the agent can see only part of the state, outcomes have randomness, and the environment changes while the agent is deciding is classified as:

- A) Fully observable, deterministic, static
- B) Partially observable, stochastic, dynamic
- C) Partially observable, deterministic, static
- D) Fully observable, stochastic, episodic

Correct Answer: B

Explanation: "See only part" = partially observable. "Randomness" = stochastic. "Changes while deciding" = dynamic. This combination describes the hardest type of environment (like driving in traffic). Option A is the opposite on all counts. Option C is wrong because stochastic ≠ deterministic. Option D is wrong because fully observable contradicts "see only part."

Question 8

A student claims: "A rational agent always makes the perfect decision and never fails." Is this claim correct?

- A) Yes, rationality means perfection
- B) No, rationality means making the best decision given available information, which may still be wrong
- C) Yes, because rational agents use optimal algorithms
- D) No, rational agents are only reflex agents

Correct Answer: B

Explanation: Rationality means maximizing expected performance given what the agent KNOWS. If information is incomplete or wrong, the rational decision can still fail. Rationality ≠ perfection. Option A and C are wrong. Option D is wrong because rationality applies to all agent types.

Question 9

A delivery drone operates autonomously but sends live video to a human operator who can take control at any time. This level of autonomy is called:

- A) Fully autonomous
- B) Human-on-the-loop
- C) Human-in-the-loop
- D) Human-operated

Correct Answer: C

Explanation: Human-in-the-loop means the human can intervene and take control at any time. Human-on-the-loop (B) means the human monitors but does not actively intervene unless necessary. Fully autonomous (A) means no human control. Human-operated (D) means human controls everything.

Question 10

A robot vacuum cleaner builds an internal map of the room, remembers where it has cleaned, and plans a path to cover the remaining areas. Compared to a simple reflex vacuum, this robot adds which capability?

- A) Learning
- B) Goal-based planning
- C) Model-based representation
- D) Utility optimization

Correct Answer: C

Explanation: Building an internal map and remembering past states is the defining feature of a Model-Based Reflex Agent. It has an internal model of the world. While it might also plan (B), the KEY addition over a simple reflex agent is the internal model. Learning (A) would require improving over time from experience. Utility optimization (D) involves trade-offs.

Question 11

In a multi-agent system where agents negotiate to divide tasks, the mechanism by which they agree on who does what is called:

- A) Centralization
- B) Coordination

C) Competition

D) Autonomy

Correct Answer: B

Explanation: Coordination is the process by which agents organize their activities to achieve goals. Negotiation and task division are coordination mechanisms. Centralization (A) implies a single controller. Competition (C) is the opposite of cooperation. Autonomy (D) is about self-governance, not task division.

Question 12

Which of the following is NOT a characteristic of a rational agent?

A) It selects actions that maximize expected performance

B) It has access to complete and perfect information

C) It uses its percept sequence and built-in knowledge

D) It may make mistakes if information is incomplete

Correct Answer: B

Explanation: Rational agents do NOT need complete information. They make the BEST decision with what they HAVE. This is the key distinction between rationality and omniscience. Options A, C, and D are all true characteristics of rational agents.

UNIT VI: EVOLUTION OF MODERN AI

6.1 Symbolic and Statistical AI

Simple Definition

- **Symbolic AI** (also called Good Old-Fashioned AI or GOF AI) uses explicit rules, logic, and symbols to represent knowledge and reason.
- **Statistical AI** (also called Machine Learning-based AI) learns patterns from data using statistical methods rather than hand-written rules.

Why It Matters

Understanding the difference between these two approaches helps you understand why modern AI works the way it does. It also explains the historical shift from rule-based systems to data-driven systems.

Easy Analogy

- **Symbolic AI** = A strict teacher who gives you a rulebook: “If X happens, do Y.” The teacher wrote every rule.
- **Statistical AI** = A wise elder who has seen thousands of situations and says, “In my experience, when X happens, Y usually follows.” They learned from examples, not rules.

Detailed Explanation

Symbolic AI How it works:

- Humans write explicit rules, facts, and logic
- AI uses these rules to reason and make decisions
- Based on formal logic, expert systems, and knowledge representation

Characteristics:

- **Explainable:** You can ask “Why did you decide this?” and the AI shows you the rule it used.
- **No data needed:** Rules come from human experts, not data.
- **Brittle:** If a situation is not covered by rules, the AI fails.
- **Hard to scale:** Writing rules for every possible situation is impossible.

Examples:

- Expert systems (MYCIN, DENDRAL)
- Chess programs with hand-coded strategies
- Rule-based spam filters (“If email contains ‘free money’, then spam”)
- Prolog programs using logic

Statistical AI / Machine Learning How it works:

- Feed the AI large amounts of data
- The AI finds statistical patterns on its own
- Uses probability, statistics, and optimization

Characteristics:

- **Data-hungry:** Needs thousands or millions of examples
- **Flexible:** Can handle situations never explicitly programmed
- **Less explainable:** Often a “black box” — hard to say exactly WHY it decided something
- **Scalable:** More data generally means better performance

Examples:

- Neural networks and deep learning
- Spam filters that learn from user behavior
- Recommendation systems (Netflix, Amazon)
- Speech and image recognition

The Shift from Symbolic to Statistical 1950s–1980s: Symbolic AI dominated

- Scientists believed intelligence came from reasoning and logic
- Expert systems were the peak achievement
- AI winter occurred because rule-based systems could not handle real-world complexity

1990s–2000s: Statistical methods rose

- More data became available
- Computing power increased
- Machine learning algorithms (SVM, decision trees, neural networks) proved more practical
- IBM's Deep Blue (1997) used a mix of rules AND statistical evaluation

2010s–Present: Statistical AI dominates

- Deep learning revolution
- Big data + GPUs + algorithms = breakthroughs
- Symbolic AI is still used in specific domains (legal reasoning, theorem proving)
- **Neuro-Symbolic AI** is emerging — combining both approaches

Practical Example

Example: Recognizing a Cat in a Photo

Symbolic AI approach:

- Write rules: “If pointy ears AND whiskers AND fur AND tail, then cat”
- Problem: What if the cat is sleeping and ears are hidden? What if it is a cartoon cat? Rules fail.

Statistical AI approach:

- Show the AI 1 million cat photos and 1 million non-cat photos
- The AI learns what “cat-ness” looks like statistically
- It recognizes cats in new photos even if they are in weird positions

Common Mistakes Students Make

- **✗** Thinking symbolic AI is dead. It is still used where explainability matters (law, medicine).
- **✗** Believing statistical AI is always better. For small, well-defined problems, symbolic AI is faster and more reliable.
- **✗** Not understanding that modern AI often combines both (neuro-symbolic AI).

Quick Summary

- Symbolic AI = hand-written rules + logic, explainable but brittle
- Statistical AI = learns from data, flexible but less explainable
- Historical shift: Symbolic (1950s–1980s) → Statistical (1990s–present)
- Neuro-Symbolic AI combines both approaches

6.2 Machine Learning: Overview and Types

Simple Definition

Machine Learning (ML) is a type of AI where computers learn patterns from data instead of following explicit programming. There are three main types: **Supervised**, **Unsupervised**, and **Reinforcement Learning**.

Why It Matters

ML is the engine behind most modern AI applications. Understanding the three types helps you know which approach fits which problem.

Easy Analogy

Imagine three different ways to learn cooking:

- **Supervised Learning** = You follow a recipe book. Each recipe tells you ingredients (input) and the final dish (output). You learn the mapping.
- **Unsupervised Learning** = You are given a basket of random ingredients and must figure out how to group them (vegetables together, spices together) without any recipe.
- **Reinforcement Learning** = You cook by trial and error. If the dish tastes good, you get a reward (praise). If it tastes bad, you adjust. Over time, you learn the best recipes through experience.

Detailed Explanation

Supervised Learning How it works:

- Training data includes BOTH input (features) and correct output (labels)
- The AI learns a mapping: Input $\rightarrow$ Output
- Like a student studying with an answer key

Two main tasks:

1. **Classification:** Predict a category
 - Example: Is this email spam or not spam? (Categories: spam, not spam)
 - Example: What digit is in this image? (Categories: 0, 1, 2, ..., 9)
2. **Regression:** Predict a continuous number
 - Example: What will the temperature be tomorrow? (Any number, not a category)
 - Example: What is the price of this house? (Any rupee amount)

Common Algorithms:

- Linear Regression, Logistic Regression
- Decision Trees, Random Forests
- Support Vector Machines (SVM)
- Neural Networks (Deep Learning)

Example: You have a dataset of 1000 houses with features (size, location, bedrooms) and prices (labels). The ML model learns to predict prices for new houses.

Unsupervised Learning How it works:

- Training data has ONLY inputs, NO labels
- The AI finds hidden patterns, groupings, or structures
- Like a student given a pile of unlabeled photos and asked to sort them

Two main tasks:

1. **Clustering:** Group similar data points together
 - Example: Group customers by shopping behavior
 - Example: Segment an image into different objects
2. **Dimensionality Reduction:** Reduce the number of features while keeping important information
 - Example: Compress a photo without losing quality
 - Example: Visualize high-dimensional data in 2D or 3D

Common Algorithms:

- K-Means Clustering
- Hierarchical Clustering
- Principal Component Analysis (PCA)
- Autoencoders

Example: A supermarket has purchase data but does not know customer types. Unsupervised learning groups customers into: “Budget Shoppers,” “Premium Shoppers,” “Health Conscious,” etc.

Reinforcement Learning (RL) How it works:

- An **agent** interacts with an **environment**
- Takes **actions** and receives **rewards** or **penalties**
- Learns a **policy** (strategy) to maximize total reward over time
- Like training a dog with treats

Key Terms:

- **Agent:** The learner/decision-maker
- **Environment:** The world the agent interacts with
- **State:** Current situation
- **Action:** What the agent can do
- **Reward:** Feedback signal (positive or negative)
- **Policy:** The strategy the agent learns

Example: AlphaGo learned to play Go by playing millions of games against itself. Winning gave positive reward; losing gave negative reward. It discovered strategies no human had found.

Applications:

- Game playing (chess, Go, video games)

- Robot control
- Autonomous driving
- Resource management (data center cooling)

Comparison Table

Feature	Supervised	Unsupervised	Reinforcement
Data	Labeled (input + output)	Unlabeled (input only)	No dataset; learns from interaction
Goal	Predict output	Find hidden patterns	Maximize reward
Feedback	Immediate (correct answer)	None (self-discovery)	Delayed (reward after actions)
Example Task	Spam detection	Customer segmentation	Game playing
Like	Student with answer key	Student sorting unlabeled items	Dog learning tricks with treats

Common Mistakes Students Make

- **✗** Using supervised learning when you do not have labels. Then you need unsupervised or semi-supervised learning.
- **✗** Thinking reinforcement learning is just “learning with feedback.” The feedback is DELAYED and sparse, which makes RL uniquely difficult.
- **✗** Confusing clustering with classification. Clustering is unsupervised (no labels); classification is supervised (has labels).

Quick Summary

- Supervised Learning = learns from labeled data (input → known output)
 - Classification = predict category; Regression = predict number
- Unsupervised Learning = finds patterns in unlabeled data
 - Clustering = group similar items; Dimensionality reduction = compress data
- Reinforcement Learning = learns by trial and error with rewards/penalties
 - Agent + Environment + Actions + Rewards = Policy

6.3 Deep Learning: Introduction and Applications

Simple Definition

Deep Learning is a subset of Machine Learning that uses artificial neural networks with MANY layers (hence “deep”) to learn complex patterns from large amounts of data.

Why It Matters

Deep Learning powers the most impressive AI achievements: face recognition, language translation, self-driving cars, and generative AI. Understanding it helps you grasp how modern AI works.

Easy Analogy

Imagine a factory assembly line with many workers:

- **Worker 1** looks at raw materials and picks out obvious features (color, shape)
- **Worker 2** combines those features (red + round = maybe apple)
- **Worker 3** combines further (apple + on tree + red = definitely apple)
- **Worker 10** makes the final decision: “This is an apple.”

Each worker is a “layer” in the neural network. Deep learning has MANY such workers (layers), each extracting more complex features.

Detailed Explanation

What is a Neural Network? A neural network is inspired by the human brain:

- **Neurons:** Small computing units
- **Layers:** Groups of neurons organized in levels
- **Connections:** Links between neurons with weights (importance)

Simple Neural Network (1 hidden layer):

Input Layer → Hidden Layer → Output Layer

Deep Neural Network (many hidden layers):

Input → Hidden 1 → Hidden 2 → Hidden 3 → ... → Hidden N → Output

How Deep Learning Works (Simplified)

1. **Input:** Data enters (image pixels, text words, audio samples)
2. **Forward Pass:** Data flows through layers. Each layer transforms the data slightly.
3. **Feature Extraction:** Early layers detect simple features (edges in images). Deeper layers detect complex features (faces, objects).
4. **Output:** Final layer makes a prediction (“This is a cat”)
5. **Error Calculation:** Compare prediction with actual answer
6. **Backpropagation:** Send error backward through the network, adjusting weights
7. **Repeat:** Do this millions of times until the network learns

Visual Description: Imagine a sieve with many layers. The first layer catches big stones. The next catches smaller stones. By the end, only sand remains. Each layer filters out more detail.

Types of Deep Learning Architectures

1. **Convolutional Neural Networks (CNNs)**
 - Best for: Images and videos
 - Uses “filters” to scan images and detect features
 - Example: Face recognition, medical image analysis
2. **Recurrent Neural Networks (RNNs)**
 - Best for: Sequential data (text, speech, time series)
 - Has memory of previous inputs

- Example: Language translation, stock prediction

3. Transformers

- Best for: Natural language processing
- Uses “attention mechanism” to focus on important words
- Example: ChatGPT, Google Translate, BERT

4. Generative Adversarial Networks (GANs)

- Best for: Generating realistic data
- Two networks compete: Generator creates fakes; Discriminator detects fakes
- Example: Deepfakes, AI art, synthetic data

Applications of Deep Learning

Domain	Application
Healthcare	Detecting cancer from X-rays, drug discovery
Automotive	Self-driving cars, pedestrian detection
Finance	Fraud detection, algorithmic trading
Entertainment	Content recommendation, AI-generated music
Security	Face recognition, anomaly detection
Agriculture	Crop disease detection, yield prediction
Retail	Visual search, inventory management

Practical Example

Example: Face Recognition on Your Phone

1. **Input:** Camera captures your face (millions of pixels)
2. **Layer 1–3:** Detect edges, corners, simple shapes
3. **Layer 4–7:** Detect eyes, nose, mouth positions
4. **Layer 8–10:** Detect face shape, skin texture, unique features
5. **Output Layer:** Compare with stored face data → “Match” or “No Match”

The network was trained on millions of face photos to learn what makes each face unique.

Common Mistakes Students Make

- **✗** Thinking deep learning is just “neural networks with more layers.” While true, the depth enables feature learning that shallow networks cannot do.
- **✗** Believing deep learning works with small data. It typically needs thousands to millions of examples.
- **✗** Confusing CNNs with RNNs. CNNs = images; RNNs = sequences.
- **✗** Thinking deep learning is always better. For small, structured datasets, traditional ML often wins.

Quick Summary

- Deep Learning = Machine Learning with many-layered neural networks
- Inspired by the brain: neurons, layers, connections with weights
- Key architectures: CNNs (images), RNNs (sequences), Transformers (NLP), GANs (generation)
- Needs lots of data and computing power

- Applications span healthcare, automotive, finance, entertainment, and more
-

6.4 Foundation Models and Modern AI Paradigms

Simple Definition

- **Foundation Models** are large AI models trained on vast amounts of data that can be adapted to many different tasks.
- **Modern AI Paradigms** include multimodal AI, agentic AI, edge AI, and tiny AI.

Why It Matters

Foundation models like GPT-4 and BERT have changed how AI is built. Instead of training a new model for every task, you start with a foundation model and fine-tune it. This is faster, cheaper, and more powerful.

Easy Analogy

- **Traditional AI:** You hire a new employee and train them from scratch for every job.
- **Foundation Models:** You hire a PhD graduate who already knows a LOT about the world. You just give them a short orientation for your specific task.

Detailed Explanation

Foundation Models What makes them special:

1. **Scale:** Trained on billions of words/images (internet-scale data)
2. **Generality:** One model can do many tasks (write essays, answer questions, write code)
3. **Transfer Learning:** Knowledge from one task helps with another
4. **Emergent Abilities:** Capabilities appear that were not explicitly trained

Examples:

- **GPT (Generative Pre-trained Transformer):** Text generation, ChatGPT
- **BERT:** Understanding text context, search engines
- **DALL-E / Midjourney:** Generating images from text
- **CLIP:** Connecting images and text

How they work:

1. **Pre-training:** The model reads billions of documents and learns language patterns
2. **Fine-tuning:** The model is trained on a smaller, specific dataset for a specific task
3. **Prompting:** You give the model instructions in natural language, and it performs the task

Multimodal AI Definition: AI that can understand and generate multiple types of data simultaneously — text, images, audio, video.

Example: GPT-4o can:

- Look at a photo and describe it
- Listen to your voice and respond
- Read text and generate images

Why it matters: The real world is multimodal. Humans see, hear, and read at the same time. Multi-modal AI is closer to human-like intelligence.

Agentic AI Definition: AI systems that can act autonomously to achieve goals, not just respond to prompts.

Characteristics:

- Can plan multi-step tasks
- Can use tools (search the web, run code, send emails)
- Can reflect on its own performance and improve

Example: An AI that says “I will research the best laptops under 50,000 rupees, compare specs, read reviews, and give you a ranked list.” It breaks the task into steps and executes them.

Edge AI Definition: Running AI models directly on local devices (phones, cameras, IoT devices) instead of sending data to the cloud.

Advantages:

- **Low latency:** No internet delay
- **Privacy:** Data stays on your device
- **Reliability:** Works without internet
- **Cost:** Less cloud computing needed

Example: Face unlock on your phone. The AI runs entirely on your phone without sending your face to a server.

Tiny AI / TinyML Definition: Running AI on extremely small, low-power devices like microcontrollers.

Applications:

- Smartwatches detecting falls
- Hearing aids filtering noise
- Sensors detecting machine vibrations
- Agricultural sensors monitoring soil moisture

Challenge: Microcontrollers have tiny memory (kilobytes) and slow processors. Models must be compressed dramatically.

Practical Example

Example: A Smart Factory Using Modern AI Paradigms

- **Foundation Model:** A pre-trained vision model that recognizes objects

- **Multimodal:** The system reads text labels on boxes AND sees the boxes
- **Agentic:** The AI plans the optimal packing strategy and adjusts if items are missing
- **Edge AI:** Cameras on the factory floor run AI locally to detect defects in real-time
- **Tiny AI:** Small sensors on machines predict maintenance needs using minimal power

Common Mistakes Students Make

- **✗** Thinking foundation models are just “big neural networks.” Their key feature is transfer learning and generality.
- **✗** Confusing multimodal with multi-task. Multimodal = multiple data types. Multi-task = multiple jobs.
- **✗** Believing edge AI is less powerful. It is optimized for speed and privacy, not raw capability.
- **✗** Thinking agentic AI is AGI. Agentic AI is still narrow — it just plans steps within its domain.

Quick Summary

- Foundation Models = large, general-purpose models trained on internet-scale data, adaptable to many tasks
- Multimodal AI = processes text, image, audio, video together
- Agentic AI = autonomously plans and executes multi-step tasks
- Edge AI = runs on local devices for speed and privacy
- Tiny AI = runs on microcontrollers with extreme resource constraints

6.5 Generative AI and Large Language Models

Simple Definition

- **Generative AI** creates new content: text, images, music, code, videos.
- **Large Language Models (LLMs)** are a type of generative AI trained on massive text data to understand and generate human language.

Why It Matters

Generative AI is the most visible AI technology today. ChatGPT, Gemini, Claude, DALL-E — these tools are changing how we work, study, and create. Understanding how they work and their limitations is essential.

Easy Analogy

- **Traditional AI** = A librarian who finds existing books for you.
- **Generative AI** = A writer who composes a brand-new story just for you, based on everything they have ever read.

Detailed Explanation

What are Large Language Models (LLMs)? Key Characteristics:

1. **Large:** Billions or trillions of parameters (think of parameters as “knobs” the model adjusts during learning)
2. **Pre-trained:** Learned language patterns from billions of web pages, books, and articles

3. **Transformer-based:** Uses the transformer architecture with attention mechanism
4. **Autoregressive:** Generates text one word at a time, predicting the next most likely word

How LLMs Work (Simplified):

1. **Tokenization:** Text is broken into small pieces called tokens (words or subwords)
 - “Hello world” → [“Hello”, “ world”]
2. **Embedding:** Each token is converted to a number vector
 - “Hello” → [0.2, -0.5, 1.3, ...]
3. **Attention:** The model figures out which words relate to which other words
 - In “The cat sat on the mat because it was tired,” the model learns “it” refers to “cat”
4. **Prediction:** The model predicts the next token based on all previous tokens
 - Input: “The capital of France is”
 - Predicted next token: “Paris”
5. **Generation:** Repeat prediction to generate entire sentences, paragraphs, or essays

Prompt Engineering Definition: The art of writing clear, effective instructions (prompts) to get the best output from an LLM.

Why it matters: LLMs are sensitive to how you phrase your request. A well-written prompt gives much better results.

Basic Prompt Design Techniques:

1. **Be Specific:**
 - Bad: “Write about AI”
 - Good: “Write a 300-word summary of how AI is used in Indian agriculture, suitable for college students.”
2. **Give Context:**
 - Bad: “Explain this code”
 - Good: “I am a beginner in Python. Please explain what this function does and suggest improvements.”
3. **Use Examples (Few-Shot Prompting):**
 - “Translate English to Hindi: Cat -> Billi. Dog -> Kutta. Elephant -> ?”
4. **Assign a Role:**
 - “You are an experienced physics teacher. Explain quantum mechanics to a 10-year-old.”
5. **Chain of Thought:**
 - “Solve this step by step: If a train travels 60 km/h for 2 hours and then 40 km/h for 3 hours, what is the average speed?”

Retrieval-Augmented Generation (RAG) Problem: LLMs only know what they were trained on. They do not know your company’s private documents or today’s news.

Solution: RAG

1. User asks a question
2. The system searches a knowledge base (your documents, databases)
3. Relevant information is retrieved
4. The LLM uses this retrieved information to generate an accurate, up-to-date answer

Example: A company chatbot answers employee questions using the company’s internal policy documents. Without RAG, the LLM would make up policies.

Hallucination in LLMs Definition: When an LLM generates confident-sounding but FALSE or NONSENSICAL information.

Why it happens:

- LLMs predict what SOUNDS right, not what IS right
- They have no real-world understanding or fact-checking
- They “hallucinate” facts, citations, and even people

Examples of Hallucination:

- Claiming a research paper exists that does not
- Inventing historical events
- Making up legal precedents
- Giving wrong medical advice

Mitigation Strategies:

1. **RAG:** Ground responses in verified documents
2. **Fact-checking layers:** Cross-check claims against trusted databases
3. **Temperature control:** Lower “temperature” makes output more conservative and factual
4. **Human review:** Critical outputs should always be verified by humans
5. **Prompt engineering:** Ask the model to “only use the provided information” or “say I don’t know if unsure”

Practical Example

Example: Using ChatGPT for Exam Preparation

Bad Prompt: “Tell me about AI” → Generic, unfocused response

Good Prompt: “You are an AI professor preparing students for a university exam. Explain the difference between BFS and DFS search algorithms. Use simple language, include a real-world analogy, and list 3 key differences in a table.” → Focused, structured, useful response

Common Mistakes Students Make

- **✗** Trusting LLM outputs blindly. Always verify facts, especially for academics.
- **✗** Writing vague prompts. Specificity dramatically improves results.
- **✗** Thinking LLMs “understand” like humans. They are statistical pattern matchers, not thinkers.
- **✗** Confusing RAG with fine-tuning. RAG retrieves documents at query time; fine-tuning changes the model itself.

Quick Summary

- Generative AI creates new content; LLMs are text-generating foundation models
- LLMs use transformers, attention, and next-token prediction

- Prompt engineering = writing effective instructions to get better outputs
 - RAG = retrieving real documents to ground LLM responses in facts
 - Hallucination = confident false outputs; mitigate with RAG, fact-checking, and human review
-

6.6 Responsible and Explainable AI

Simple Definition

- **Explainable AI (XAI)** makes AI decisions understandable to humans.
- **Responsible AI** ensures AI is fair, transparent, private, secure, and ethical.

Why It Matters

As AI makes more important decisions (loans, medical diagnoses, hiring), we **MUST** understand **WHY** it decided what it did. Unfair or opaque AI can harm people and society.

Easy Analogy

- **Explainable AI** = A teacher who shows their work in math problems. You can see **HOW** they got the answer.
- **Responsible AI** = A teacher who grades fairly, respects student privacy, admits mistakes, and follows school rules.

Detailed Explanation

Explainable AI (XAI) The Black Box Problem: Deep learning models are “black boxes” — they make decisions but cannot explain why. This is dangerous in:

- Medicine: “Why did you diagnose cancer?”
- Finance: “Why was my loan rejected?”
- Law: “Why is this person a flight risk?”

XAI Techniques:

1. **Feature Importance:** Shows which inputs mattered most
 - Example: “Loan rejected because: credit score (40%), income (30%), existing loans (20%)”
2. **LIME (Local Interpretable Model-agnostic Explanations):**
 - Explains individual predictions by approximating the model locally with a simple, interpretable model
3. **SHAP (SHapley Additive exPlanations):**
 - Uses game theory to fairly distribute “credit” for a prediction among all features
4. **Attention Visualization:**
 - In NLP, shows which words the model focused on
 - Example: Highlighting “fever” and “cough” as key words in a diagnosis
5. **Counterfactual Explanations:**
 - “If your income were 10,000 higher, your loan would have been approved.”

Fairness, Bias, and Transparency Types of Bias:

1. **Data Bias:** Training data does not represent reality fairly

- Example: A hiring AI trained mostly on male resumes discriminates against women
2. **Algorithmic Bias:** The algorithm itself amplifies unfair patterns
 - Example: A loan AI gives lower scores to certain zip codes, perpetuating redlining
 3. **Interaction Bias:** The AI learns biased behavior from interacting with biased humans
 - Example: Chatbots learning toxic language from internet users

Fairness Metrics:

- **Demographic Parity:** Equal positive prediction rates across groups
- **Equal Opportunity:** Equal true positive rates across groups
- **Individual Fairness:** Similar individuals should receive similar treatment

Mitigating Bias:

- Audit training data for representation
- Remove sensitive attributes (race, gender) from training when irrelevant
- Test models on diverse populations
- Include diverse teams in AI development

Responsible AI Principles

1. **Privacy:**
 - AI should not leak personal information
 - Techniques: Data anonymization, differential privacy, federated learning
2. **Security:**
 - AI systems should be protected from attacks
 - Adversarial attacks: Slightly modified inputs that fool AI (e.g., stickers on a stop sign making AI think it is a speed limit)
3. **AI Governance:**
 - Organizations need rules for AI development and deployment
 - Includes ethics boards, review processes, and accountability
4. **Ethical and Societal Implications:**
 - Job displacement from automation
 - Surveillance and privacy erosion
 - Autonomous weapons
 - Deepfakes and misinformation
 - Environmental impact of training large models

Practical Example

Example: Loan Approval AI

Without XAI:

- AI rejects a loan application
- Applicant asks: “Why?”
- Bank says: “The AI decided. We don’t know exactly why.”
- Result: Angry customer, potential lawsuit, regulatory trouble

With XAI:

- AI rejects a loan application
- Explanation: “Rejected because: (1) Credit score below threshold, (2) Debt-to-income ratio too high, (3) Short employment history.”
- Applicant understands and knows what to improve
- Bank can audit for bias: “Are we rejecting more women than men with similar profiles?”

Common Mistakes Students Make

- ❌ Thinking XAI is only about making pretty charts. It is about TRUST and ACCOUNTABILITY.
- ❌ Believing AI is objective. AI learns from human data, and humans are biased.
- ❌ Ignoring privacy in AI. Collecting more data is not always better if it violates privacy.
- ❌ Thinking ethics is someone else’s problem. Every AI developer has ethical responsibilities.

Quick Summary

- Explainable AI (XAI) makes AI decisions understandable — essential for trust and accountability
 - Techniques: Feature importance, LIME, SHAP, attention visualization, counterfactuals
 - Bias comes from data, algorithms, and human interaction; must be actively mitigated
 - Responsible AI principles: Fairness, Privacy, Security, Governance, Ethics
 - AI has real societal impacts: jobs, surveillance, misinformation, environment
-

6.7 AI Ecosystem and Future Trends

Simple Definition

The **AI ecosystem** is the collection of tools, frameworks, platforms, and communities that enable AI development. **Future trends** include AGI, sustainable AI, and new paradigms.

Why It Matters

Knowing the ecosystem helps you understand how real AI projects are built. Knowing future trends helps you prepare for where the field is going.

Easy Analogy

The AI ecosystem is like the automobile industry:

- **Car manufacturers** = AI model developers (OpenAI, Google)
- **Engine makers** = Framework developers (TensorFlow, PyTorch)
- **Gas stations** = Cloud providers (AWS, Azure, GCP)
- **Driving schools** = Learning platforms (Coursera, YouTube)
- **Mechanics** = MLOps engineers who maintain deployed models

Detailed Explanation

The AI Development Ecosystem 1. Python Ecosystem

- Python is the dominant language for AI
- Simple syntax, huge library support
- Key libraries: NumPy (math), Pandas (data), Matplotlib (plots), Scikit-learn (ML)

2. Deep Learning Frameworks

TensorFlow (Google):

- Developed by Google Brain team
- Good for production deployment
- Supports mobile and edge deployment (TensorFlow Lite)
- Used in: Google Search, Photos, Translate

PyTorch (Meta/Facebook):

- More Pythonic and flexible
- Preferred by researchers
- Dynamic computation graphs
- Used in: Meta's AI research, many academic papers

3. Model Hubs and Platforms

Hugging Face:

- "GitHub for ML models"
- Hosts thousands of pre-trained models
- Easy to download and use models with a few lines of code
- Key libraries: Transformers, Datasets, Tokenizers

4. Local LLM Tools

Ollama:

- Run LLMs locally on your laptop
- Privacy-friendly (data does not leave your machine)
- Supports models like Llama, Mistral, Gemma

5. AI Application Frameworks

LangChain:

- Framework for building applications with LLMs
- Connects LLMs to databases, APIs, search engines
- Enables RAG, agents, and complex workflows

6. Vector Databases

What they are: Databases optimized for storing and searching high-dimensional vectors (embeddings)

Why they matter: When you ask an LLM a question, RAG systems convert your question and documents into vectors and find the most similar ones.

Examples: Pinecone, Weaviate, Chroma, FAISS

Visual Description: Imagine a library where books are organized not by title, but by “meaning.” Books about similar topics are placed near each other. A vector database does this for AI — it finds documents with similar meaning to your query.

AI Development Workflow

1. **Problem Definition:** What are we solving?
2. **Data Collection:** Gather and label data
3. **Exploration:** Understand data using Python/Pandas
4. **Model Selection:** Choose from Hugging Face or build with TensorFlow/PyTorch
5. **Training:** Train on GPUs (cloud or local)
6. **Evaluation:** Test performance
7. **Deployment:** Put into production (cloud, edge, or mobile)
8. **Monitoring:** Track performance and drift
9. **Iteration:** Retrain with new data

Future Trends in AI

1. **Artificial General Intelligence (AGI):**
 - AI that can perform any intellectual task a human can
 - Currently does not exist
 - Debate: Will it happen? When? Is it safe?
 - Companies like OpenAI and DeepMind are pursuing it
2. **Sustainable AI / Green AI:**
 - Training large models consumes enormous energy
 - Focus on: smaller efficient models, renewable energy for data centers, model compression
 - Example: Distilling a large model into a tiny model that runs on phones
3. **Neuro-Symbolic AI:**
 - Combining neural networks (statistical) with symbolic reasoning (logic)
 - Goal: Get the learning power of deep learning + the explainability of symbolic AI
4. **AI Regulation:**
 - EU AI Act, India’s AI strategy, US executive orders
 - Focus on: safety, transparency, accountability, preventing harm
5. **Quantum Machine Learning:**
 - Using quantum computers to speed up ML algorithms
 - Still experimental but promising for optimization and simulation
6. **Personalized AI:**
 - AI models fine-tuned to individual users
 - Your AI assistant that knows your preferences, schedule, and style

Practical Example

Example: Building a Student Assistant Chatbot

Tools used:

1. **Python** for coding
2. **Hugging Face** to download a pre-trained LLM (like Llama or Mistral)
3. **Ollama** to run the model locally
4. **LangChain** to connect the LLM to the university's course database
5. **Chroma** (vector database) to store and retrieve course materials for RAG
6. **TensorFlow/PyTorch** (if fine-tuning is needed)

Workflow:

- Student asks: "When is the AI exam?"
- LangChain sends query to Chroma vector DB
- Chroma finds relevant documents (academic calendar, course page)
- LLM generates answer using retrieved information
- Response: "The AI exam is on March 15th, 10 AM, Block 33."

Common Mistakes Students Make

- **✗** Thinking you need to build models from scratch. Most projects use pre-trained models from Hugging Face.
- **✗** Ignoring the deployment side. A model in a Jupyter notebook is not a product.
- **✗** Believing bigger models are always better. Smaller, efficient models often win in production.
- **✗** Not considering sustainability. Training a huge model for a simple task is wasteful.

Quick Summary

- AI Ecosystem: Python → TensorFlow/PyTorch → Hugging Face → Ollama/LangChain → Vector DBs
- Development workflow: Problem → Data → Model → Train → Evaluate → Deploy → Monitor
- Future trends: AGI, Sustainable AI, Neuro-Symbolic AI, AI Regulation, Quantum ML, Personalized AI
- Real projects combine multiple tools from the ecosystem
- Sustainability and ethics are becoming as important as accuracy

UNIT VI: KEY POINTS TO REMEMBER

- Symbolic AI = hand-written rules and logic; Statistical AI = learning from data
- ML has three types: Supervised (labeled data), Unsupervised (unlabeled data), Reinforcement (rewards/penalties)
- Supervised: Classification (categories) and Regression (numbers)
- Unsupervised: Clustering (grouping) and Dimensionality Reduction

- Reinforcement: Agent + Environment + Actions + Rewards = Policy
 - Deep Learning = neural networks with many layers; architectures: CNN (images), RNN (sequences), Transformers (NLP), GANs (generation)
 - Foundation Models = large, general-purpose, adaptable to many tasks via fine-tuning or prompting
 - Multimodal AI = processes multiple data types (text, image, audio)
 - Agentic AI = autonomously plans and executes multi-step tasks
 - Edge AI = runs on local devices; Tiny AI = runs on microcontrollers
 - LLMs use transformers, attention, and next-token prediction
 - Prompt engineering = writing effective instructions for LLMs
 - RAG = retrieve documents + generate answers = more accurate, up-to-date responses
 - Hallucination = confident false outputs; mitigate with RAG, fact-checking, human review
 - XAI = explainable AI; techniques: feature importance, LIME, SHAP, attention maps, counterfactuals
 - Bias sources: data, algorithms, human interaction; must be actively mitigated
 - Responsible AI: Fairness, Privacy, Security, Governance, Ethics
 - AI Ecosystem: Python, TensorFlow, PyTorch, Hugging Face, Ollama, LangChain, Vector DBs
 - Future trends: AGI, Sustainable AI, Neuro-Symbolic AI, Regulation, Quantum ML
-

UNIT VI: SELF-CHECK QUESTIONS

1. What is the main difference between symbolic AI and statistical AI?
 2. Name the three types of machine learning and give one example of each.
 3. What is the difference between classification and regression?
 4. What makes a model a “foundation model”?
 5. What is RAG, and why is it important for LLMs?
 6. What is hallucination, and name two ways to reduce it.
 7. Why is Explainable AI (XAI) important in healthcare?
 8. Name three tools in the modern AI ecosystem and what they are used for.
 9. What is the difference between Edge AI and Cloud AI?
 10. What is reinforcement learning, and what are its key components?
-

UNIT VI: SCENARIO-BASED MCQs

Question 1

A company has 100,000 customer records with labels indicating whether each customer churned (left) or stayed. They want to build an AI that predicts whether a NEW customer will churn. Which type of machine learning is this?

- A) Unsupervised Learning
- B) Reinforcement Learning
- C) Supervised Learning
- D) Deep Learning only

Correct Answer: C

Explanation: The data has labels (churned/stayed), and the goal is to predict a category for new data. This is classic Supervised Learning (specifically, classification). Unsupervised learning (A) has no labels. Reinforcement learning (B) learns from rewards, not labeled datasets. Deep learning (D) is a technique, not a learning type — you could use deep learning WITHIN supervised learning.

Question 2

A supermarket has purchase data but no customer categories. They want the AI to automatically discover groups of customers with similar buying habits. Which technique should they use?

- A) Linear Regression
- B) K-Means Clustering
- C) Decision Tree Classification
- D) Q-Learning

Correct Answer: B

Explanation: Grouping unlabeled data into clusters is K-Means Clustering, an unsupervised learning technique. Linear regression (A) predicts continuous values. Decision trees (C) require labels. Q-Learning (D) is a reinforcement learning algorithm.

Question 3

A game AI learns to play Mario by trying different moves. When Mario collects coins, the AI gets positive points. When Mario falls in a pit, it gets negative points. Over thousands of games, the AI improves its score. This is an example of:

- A) Supervised Learning with labeled game frames
- B) Unsupervised Learning discovering game patterns
- C) Reinforcement Learning with rewards and penalties
- D) Transfer Learning from other games

Correct Answer: C

Explanation: The AI learns through trial and error with rewards (coins) and penalties (falling). This is the definition of Reinforcement Learning. There are no labels (A is wrong), no pattern discovery in unlabeled data (B is wrong), and no mention of transferring from other games (D is wrong).

Question 4

A startup wants to build an AI assistant for their company but does not have enough data or budget to train a model from scratch. They download a pre-trained language model and fine-tune it on their company's documents. The pre-trained model is an example of:

- A) A Neural Network
- B) A Foundation Model
- C) An Expert System
- D) A Vector Database

Correct Answer: B

Explanation: A pre-trained, general-purpose model that can be adapted to specific tasks is a Foundation Model. While it IS a neural network (A), "foundation model" is the more precise term. Expert systems (C) use rules, not pre-trained models. Vector databases (D) store embeddings, not models.

Question 5

A hospital deploys an AI that diagnoses diseases from X-rays. The doctors demand to know WHY the AI flagged a particular area as cancerous. The hospital implements a system that highlights the specific image regions that influenced the AI's decision. This is an example of:

- A) RAG
- B) XAI (Explainable AI)
- C) Edge AI
- D) Transfer Learning

Correct Answer: B

Explanation: Making AI decisions understandable through visual explanations (highlighting regions) is Explainable AI (XAI). RAG (A) retrieves documents. Edge AI (C) runs on local devices. Transfer learning (D) reuses pre-trained models.

Question 6

A company chatbot answers employee questions about company policies. Instead of relying on its training data (which may be outdated), the chatbot searches the company's internal policy database and uses those documents to generate answers. This technique is called:

- A) Fine-tuning

B) Retrieval-Augmented Generation (RAG)

C) Prompt Engineering

D) Hallucination

Correct Answer: B

Explanation: Retrieving real documents and using them to ground the AI's response is RAG. Fine-tuning (A) changes the model weights. Prompt engineering (C) is about writing good instructions. Hallucination (D) is making things up — the OPPOSITE of what RAG prevents.

Question 7

An AI model generates a detailed research paper citation: “Smith et al. (2023), Journal of Advanced Medicine, Vol. 45, pp. 112-120.” However, no such paper exists. This is an example of:

A) RAG failure

B) Hallucination

C) Overfitting

D) Underfitting

Correct Answer: B

Explanation: Generating a confident-sounding but false citation is classic LLM hallucination. RAG failure (A) would occur if retrieval failed. Overfitting (C) is when a model memorizes training data. Underfitting (D) is when a model is too simple.

Question 8

A developer wants to run a language model on a laptop without sending data to the internet, ensuring complete privacy. Which tool is MOST suitable?

A) Hugging Face API

B) OpenAI API

C) Ollama

D) Google Cloud AI

Correct Answer: C

Explanation: Ollama is designed to run LLMs locally on personal computers, keeping all data private. Hugging Face (A), OpenAI (B), and Google Cloud (D) all involve sending data to external servers.

Question 9

A facial recognition system performs well on light-skinned faces but has high error rates on dark-skinned faces. This is primarily an issue of:

- A) Hallucination
- B) Data Bias
- C) Overfitting
- D) Underfitting

Correct Answer: B

Explanation: The model likely was trained on a dataset with mostly light-skinned faces, so it performs poorly on underrepresented groups. This is data bias. Hallucination (A) is about false generation. Overfitting (C) and underfitting (D) are general model performance issues, not specifically about demographic disparity.

Question 10

A company uses a large neural network to process images on smartphones directly, without uploading photos to the cloud. This approach is called:

- A) Cloud AI
- B) Edge AI
- C) Distributed AI
- D) Quantum AI

Correct Answer: B

Explanation: Running AI directly on local devices (smartphones) is Edge AI. Cloud AI (A) sends data to remote servers. Distributed AI (C) involves multiple machines working together. Quantum AI (D) uses quantum computers.

Question 11

Which of the following best describes the transformer architecture used in LLMs?

- A) It processes words one at a time in sequence, like RNNs
- B) It uses attention mechanisms to relate different words regardless of their distance
- C) It only works for image recognition tasks
- D) It requires labeled data for every possible sentence

Correct Answer: B

Explanation: Transformers use “self-attention” to relate any word to any other word in a sentence, regardless of distance. This is why they understand context so well. Option A describes RNNs. Option C is wrong — transformers are primarily for text. Option D is wrong — they learn from unlabeled text during pre-training.

Question 12

A researcher is comparing two approaches: (1) Symbolic AI with hand-written medical rules, and (2) Deep Learning trained on millions of X-rays. Which statement is TRUE?

- A) Approach 1 is always more accurate
- B) Approach 2 is always more explainable
- C) Approach 1 is more explainable but less scalable; Approach 2 is more scalable but less explainable
- D) Both approaches require the same amount of data

Correct Answer: C

Explanation: Symbolic AI (rules) is explainable (you can see the rules) but hard to scale (writing rules for every case is impossible). Deep Learning scales well with data but is a black box. Option A is wrong — DL is often more accurate. Option B is wrong — symbolic AI is more explainable. Option D is wrong — symbolic AI needs little data; DL needs lots.

FULL PRACTICE TEST: 25 SCENARIO-BASED MCQs

Instructions: This test simulates the LPU Continuous Assessment (CA) format. Each question presents a real-world scenario. Read carefully, think about the concepts from all six units, and choose the best answer. After each question, the correct answer and detailed explanation are provided.

Time: 45 minutes

Total Marks: 25 (1 mark per question)

Question 1

A city traffic department installs an AI system at intersections. The system uses cameras to count vehicles and adjusts green light timing in real-time. It remembers traffic patterns from the past hour to predict congestion. Which combination of AI type and agent type best describes this system?

- A) Reactive Machine, Simple Reflex Agent

- B) Limited Memory AI, Model-Based Reflex Agent
- C) Theory of Mind AI, Goal-Based Agent
- D) General AI, Utility-Based Agent

Correct Answer: B

Explanation: The system remembers past hour traffic patterns = Limited Memory AI. It uses this internal model to make decisions = Model-Based Reflex Agent. It is NOT reactive (A) because it uses memory. Theory of Mind (C) and General AI (D) do not exist yet.

Question 2

A student is solving the water jug problem (4-liter and 3-liter jugs, need exactly 2 liters). She represents each possible amount of water in both jugs as a state and draws arrows for possible pour/fill/empty actions. This representation is called:

- A) A Semantic Network
- B) A Production System
- C) State-Space Representation
- D) A Frame Structure

Correct Answer: C

Explanation: Representing all possible situations (water amounts) as nodes and actions as edges is State-Space Representation. Semantic networks (A) represent concepts and relationships. Production systems (B) use if-then rules. Frames (D) use slots and fillers.

Question 3

An AI uses the formula $f(n) = g(n) + h(n)$ where $g(n)$ is the distance traveled so far and $h(n)$ is the straight-line distance to the destination. The AI always expands the node with the lowest $f(n)$. Which algorithm is this?

- A) Breadth First Search
- B) Best First Search
- C) A* Search
- D) Hill Climbing

Correct Answer: C

Explanation: $f(n) = g(n) + h(n)$ is the defining formula of A* Search. BFS (A) does not use heuristics. Best First (B) uses only $h(n)$. Hill Climbing (D) does not use this formula.

Question 4

A medical expert system uses rules like: “IF high_fever AND dry_cough AND loss_of_taste THEN possible_COVID.” The system starts with patient symptoms and applies rules to reach a diagnosis. This reasoning method is:

- A) Backward Chaining
- B) Forward Chaining
- C) Bayesian Inference
- D) Depth First Search

Correct Answer: B

Explanation: Starting with facts (symptoms) and applying rules to reach a conclusion is Forward Chaining. Backward chaining (A) starts with the goal (diagnosis) and works backward. Bayesian inference (C) uses probabilities. DFS (D) is a search algorithm.

Question 5

In First-Order Predicate Logic, the statement “All birds can fly” is written as $\forall x \text{ Bird}(x) \rightarrow \text{CanFly}(x)$. The symbol $\forall$ represents:

- A) There exists
- B) For all
- C) If and only if
- D) Therefore

Correct Answer: B

Explanation: $\forall$ is the universal quantifier meaning “for all” or “for every.” $\exists$ means “there exists.” $\leftrightarrow$ means “if and only if.” $\therefore$ means “therefore.”

Question 6

A spam filter calculates $P(\text{Spam} \mid \text{“free”, “money”, “click”})$ using the probabilities of each word appearing in spam emails. This probabilistic approach is based on:

- A) Propositional Logic
- B) Bayes’ Theorem

C) Semantic Networks

D) Frame Inheritance

Correct Answer: B

Explanation: Calculating conditional probabilities like $P(\text{Spam} \mid \text{words})$ is the core of Bayes' Theorem, specifically the Naive Bayes classifier. Propositional logic (A) uses true/false, not probabilities. Semantic networks (C) and frames (D) are knowledge representation methods, not probabilistic.

Question 7

A warehouse uses 100 robots that communicate their positions, divide picking tasks, and avoid collisions by negotiating paths. This is best described as:

A) A single autonomous agent with multiple actuators

B) A cooperative multi-agent system

C) A competitive game-theoretic system

D) A centralized expert system

Correct Answer: B

Explanation: Multiple robots working together with communication and task division is a cooperative multi-agent system. They are separate agents (not A), they cooperate (not C), and there is no central controller mentioned (not D).

Question 8

A self-driving car must choose between Route A (faster but riskier) and Route B (slower but safer). It assigns numerical scores to safety, time, and comfort, then picks the route with the highest total score. This demonstrates:

A) Goal-Based Reasoning

B) Utility-Based Reasoning

C) Simple Reflex Behavior

D) Model-Based Planning

Correct Answer: B

Explanation: Weighing multiple factors (safety, time, comfort) with numerical scores to maximize overall satisfaction is Utility-Based Reasoning. Goal-based (A) would simply try to reach the destination. Simple reflex (C) has no planning. Model-based (D) adds memory but not trade-offs.

Question 9

A company downloads a pre-trained language model from Hugging Face and adapts it to answer customer service questions for their specific products. The pre-trained model is a:

- A) Rule-Based System
- B) Foundation Model
- C) Vector Database
- D) Expert System

Correct Answer: B

Explanation: A large, pre-trained, general-purpose model that can be adapted to specific tasks is a Foundation Model. Rule-based systems (A) and expert systems (D) use hand-written rules. Vector databases (C) store embeddings, not models.

Question 10

A chatbot sometimes generates confident-sounding but completely false information, such as inventing research papers that do not exist. This phenomenon is called:

- A) Overfitting
- B) Hallucination
- C) Data Bias
- D) Underfitting

Correct Answer: B

Explanation: Generating false but confident information is hallucination — a well-known problem with LLMs. Overfitting (A) is memorizing training data. Data bias (C) is unfair representation. Underfitting (D) is poor model performance.

Question 11

An AI system for loan approval is found to reject loans from certain zip codes at higher rates, even when applicants have similar financial profiles. This is primarily an issue of:

- A) Hallucination
- B) Algorithmic Bias
- C) Overfitting
- D) Underfitting

Correct Answer: B

Explanation: Unfair treatment of certain groups (by zip code) due to patterns learned by the algorithm is algorithmic bias. Hallucination (A) is false generation. Overfitting (C) and underfitting (D) are general accuracy issues, not fairness issues.

Question 12

A student runs a neural network on a Raspberry Pi to detect faces at a remote village school gate. The model processes images locally without internet. This is an example of:

- A) Cloud AI
- B) Edge AI
- C) Distributed AI
- D) Quantum AI

Correct Answer: B

Explanation: Running AI on a local device (Raspberry Pi) without cloud connectivity is Edge AI. Cloud AI (A) sends data to remote servers. Distributed AI (C) involves multiple coordinated systems. Quantum AI (D) uses quantum computing.

Question 13

In the AI lifecycle, a team discovers that their trained model performs well on training data but poorly on new patient data. They realize the training data lacked diversity in age and ethnicity. Which lifecycle step needs improvement?

- A) Deployment
- B) Monitoring
- C) Data Collection and Preprocessing
- D) Problem Definition

Correct Answer: C

Explanation: The root cause is poor data diversity during collection. The team needs better, more representative data. Deployment (A) and monitoring (B) come after training. Problem definition (D) was likely correct (detect disease).

Question 14

A search algorithm explores all nodes at depth 1, then all nodes at depth 2, then depth 3, and so on until it finds the goal. This algorithm is:

- A) Depth First Search
- B) Breadth First Search
- C) Hill Climbing
- D) Best First Search

Correct Answer: B

Explanation: Exploring level by level (depth 1, then 2, then 3) is Breadth First Search. DFS (A) goes deep first. Hill Climbing (C) and Best First (D) use heuristics and do not systematically explore by level.

Question 15

An AI system explains its loan rejection by stating: “Your application was rejected because your debt-to-income ratio is 45% (threshold is 40%) and you have 2 recent credit defaults.” This transparency feature is an example of:

- A) RAG
- B) XAI (Explainable AI)
- C) Edge Computing
- D) Transfer Learning

Correct Answer: B

Explanation: Providing clear reasons for AI decisions is Explainable AI (XAI). RAG (A) retrieves documents for generation. Edge computing (C) is about local processing. Transfer learning (D) reuses pre-trained models.

Question 16

A robot vacuum cleaner starts with no map of the room. As it moves, it builds an internal map, remembers where it has cleaned, and plans paths to cover remaining areas. Compared to a simple reflex vacuum, this robot is best classified as:

- A) A Simple Reflex Agent
- B) A Model-Based Agent
- C) A Learning Agent
- D) A Utility-Based Agent

Correct Answer: B

Explanation: Building and using an internal map (world model) and remembering past states is the defining feature of a Model-Based Agent. It is not learning (C) because it does not improve from experience — it just maintains a model. It is not utility-based (D) because there is no mention of trade-offs.

Question 17

A company wants to build an AI that predicts house prices based on features like size, location, and number of bedrooms. They have historical data of 10,000 houses with their actual selling prices. This is a:

- A) Classification problem in Unsupervised Learning
- B) Regression problem in Supervised Learning
- C) Clustering problem in Reinforcement Learning
- D) Policy optimization problem in Deep Learning

Correct Answer: B

Explanation: Predicting a continuous number (price) from labeled historical data is a Regression problem in Supervised Learning. Classification (A) predicts categories. Clustering (C) is unsupervised. Policy optimization (D) is reinforcement learning.

Question 18

Using the PEAS framework, the “Environment” component for a medical diagnosis chatbot would include:

- A) The chatbot’s response speed and accuracy metrics
- B) Patient symptoms, medical history, and user queries
- C) The text generation algorithm and neural network
- D) The hospital’s server infrastructure

Correct Answer: B

Explanation: In PEAS, Environment is the external world the agent operates in — for a medical chatbot, this includes patient inputs (symptoms, history, queries). Option A describes Performance. Option C describes internal processing. Option D is infrastructure, not environment.

Question 19

A game AI learns to play chess by playing millions of games against itself, receiving positive rewards for wins and negative rewards for losses. It gradually discovers strategies no human

had explicitly programmed. This is:

- A) Supervised Learning with labeled game states
- B) Unsupervised Learning discovering game patterns
- C) Reinforcement Learning with self-play
- D) Transfer Learning from human games

Correct Answer: C

Explanation: Learning through trial and error with rewards/penalties from self-play is Reinforcement Learning. AlphaGo used exactly this approach. It is not supervised (A — no labels), not unsupervised (B — not finding patterns in unlabeled data), and not transfer learning (D — learning from itself, not humans).

Question 20

A frame for “Vehicle” has slots: Make, Model, Year, Color. A frame for “ElectricVehicle” inherits from “Vehicle” and adds BatteryCapacity and Range. An instance “Tesla_Model_3” inherits Make, Model, Year, Color and gets BatteryCapacity and Range. This demonstrates:

- A) Forward Chaining
- B) Frame Inheritance
- C) Bayesian Updating
- D) Semantic Network Traversal

Correct Answer: B

Explanation: Child frames inheriting properties from parent frames is Frame Inheritance. Forward chaining (A) is rule-based reasoning. Bayesian updating (C) is probabilistic. Semantic network traversal (D) involves following edges in a graph.

Question 21

A medical test for a rare disease is 95% accurate. Only 1% of the population has the disease. If a randomly selected person tests positive, what is the probability they actually have the disease? (Use Bayes’ Theorem conceptually)

- A) About 95%
- B) About 16%

C) About 50%

D) About 1%

Correct Answer: B

Explanation: This is the classic Bayes' Theorem base rate fallacy. $P(\text{Disease}|\text{Positive}) = \frac{P(\text{Positive}|\text{Disease})P(\text{Disease})}{P(\text{Positive})}$. $P(\text{Positive}) = (0.95 \times 0.01) + (0.05 \times 0.99) = 0.0095 + 0.0495 = 0.059$. So $P(\text{Disease}|\text{Positive}) = 0.0095/0.059 \approx 16\%$. Most positive tests are false positives because the disease is rare.

Question 22

A company wants to build an AI assistant that can look at product images, read their descriptions, and answer customer questions about them using both visual and textual information. This requires:

A) Only a CNN

B) Only an RNN

C) A Multimodal AI system

D) A Simple Reflex Agent

Correct Answer: C

Explanation: Processing multiple data types (images + text) simultaneously is Multimodal AI. CNNs (A) handle images. RNNs (B) handle sequences. A simple reflex agent (D) cannot handle such complex tasks.

Question 23

In a search algorithm comparison, Algorithm X always finds the shortest path but may run out of memory on large graphs. Algorithm Y uses very little memory but may find a longer path. Algorithm X is likely _____, and Algorithm Y is likely _____.

A) DFS; BFS

B) BFS; DFS

C) A*; Hill Climbing

D) Hill Climbing; Best First Search

Correct Answer: B

Explanation: BFS finds the shortest path in unweighted graphs but has high memory usage ($O(b^d)$). DFS uses low memory ($O(bm)$) but is not optimal. A* (C) is both memory-efficient (relatively) and optimal. Hill Climbing (D) is not about pathfinding.

Question 24

A developer uses LangChain to connect an LLM to a company's internal document database. When a user asks a question, the system retrieves relevant documents and uses them to generate an accurate answer. This architecture is called:

- A) Fine-Tuning
- B) Retrieval-Augmented Generation (RAG)
- C) Prompt Engineering
- D) Model Distillation

Correct Answer: B

Explanation: Retrieving documents and using them to ground LLM responses is RAG. LangChain is commonly used to build RAG systems. Fine-tuning (A) changes model weights. Prompt engineering (C) is about writing instructions. Model distillation (D) creates smaller models from larger ones.

Question 25

Which of the following represents the historical evolution of AI in the correct chronological order?

- A) Deep Learning → Expert Systems → Machine Learning → Generative AI
- B) Expert Systems → Machine Learning → Deep Learning → Generative AI
- C) Generative AI → Deep Learning → Machine Learning → Expert Systems
- D) Machine Learning → Expert Systems → Deep Learning → Generative AI

Correct Answer: B

Explanation: The correct order is: Expert Systems (1980s) → Machine Learning rise (1990s–2000s) → Deep Learning boom (2010s) → Generative AI era (2020s–present). Option A reverses early and late. Option C is completely backward. Option D places ML before expert systems, which is incorrect.

FINAL ONE-PAGE REVISION SHEET

Most Important Concepts for CSE276 CA

UNIT I: Foundations

- **AI Definition:** Machines that think, learn, and decide
- **Turing Test (1950):** Can a machine fool a human in conversation?
- **Dartmouth (1956):** Birth of “Artificial Intelligence”

- **AI Types by Capability:** ANI (all current AI) → AGI (future) → ASI (far future)
- **AI Types by Functionality:** Reactive → Limited Memory → Theory of Mind → Self-Aware
- **Hierarchy:** AI → ML → DL; Data Science overlaps but is broader
- **AI Lifecycle:** Problem → Data → Preprocessing → Training → Evaluation → Deployment → Monitoring
- **Applications:** Healthcare, Agriculture, Finance, Education, Manufacturing, Robotics, Smart Cities

UNIT II: Problem Solving

- **Problem Formulation:** Initial State + Actions + Transition Model + Goal State + Path Cost
- **State Space:** All possible states as a graph (nodes = states, edges = actions)
- **Problem Characteristics:** Decomposable? Predictable? Best solution needed? Knowledge needed? Single/Multi-agent?
- **Search Space** = all explored paths; **Solution Space** = valid paths to goal
- **Performance Measures:** Completeness, Optimality, Time Complexity, Space Complexity
- **Three Approaches:** Search-based, Knowledge-based, Learning-based

UNIT III: Search Techniques

- **Uninformed Search:** No clues
 - **BFS:** Queue, level-by-level, complete, optimal, high memory
 - **DFS:** Stack, deep-first, complete (finite), not optimal, low memory
- **Informed Search:** Uses heuristics
 - **Best First:** Greedy, only $h(n)$, not optimal
 - **Hill Climbing:** Best neighbor, gets stuck in local maxima/plateaus/ridges
 - **A*:** $f(n) = g(n) + h(n)$, complete AND optimal (if h admissible)
- **Heuristic:** Admissible = never overestimates true cost
- **Choose BFS** for shortest path (small); **DFS** for memory limits; **A*** for route planning

UNIT IV: Knowledge Representation

- **Semantic Networks:** Nodes + labeled edges (relationships)
- **Frames:** Slots + fillers, support inheritance
- **Production Systems:** Working Memory + Rule Base + Inference Engine
- **Forward Chaining:** Facts → Rules → Conclusion
- **Backward Chaining:** Goal → Rules → Verify Facts
- **Propositional Logic:** True/false with AND, OR, NOT, IMPLIES; no quantifiers
- **First-Order Logic:** Objects, predicates, functions, quantifiers (,)
- **Bayes' Theorem:** $P(A|B) = P(B|A) \cdot P(A) / P(B)$; updates beliefs with evidence
- **Key Insight:** Even accurate tests give misleading results for rare diseases

UNIT V: Intelligent Agents

- **Agent:** Perceives (sensors) + Acts (actuators)
- **PEAS:** Performance, Environment, Actuators, Sensors
- **Environment Properties:** Observable, Deterministic, Episodic/Sequential, Static/Dynamic, Discrete/Continuous, Single/Multi-agent
- **Rational Agent:** Maximizes performance given what it knows (NOT omniscient)

- **Agent Types (increasing complexity):**
 1. Simple Reflex (condition-action, no memory)
 2. Model-Based (adds internal world model)
 3. Goal-Based (adds goals and planning)
 4. Utility-Based (adds preferences and trade-offs)
 5. Learning (improves from experience)
- **Multi-Agent Systems:** Cooperative, Competitive, Mixed
- **Human-AI Collaboration:** AI assists/augments humans; future is human + AI together

UNIT VI: Modern AI

- **Symbolic AI:** Hand-written rules, explainable, brittle
- **Statistical AI:** Learns from data, flexible, less explainable
- **ML Types:** Supervised (labeled), Unsupervised (unlabeled), Reinforcement (rewards)
- **Deep Learning:** Many-layered neural networks; CNNs (images), RNNs (sequences), Transformers (NLP), GANs (generation)
- **Foundation Models:** Large, general-purpose, adaptable via fine-tuning/prompting
- **Multimodal AI:** Processes text, image, audio, video together
- **Agentic AI:** Autonomously plans and executes multi-step tasks
- **Edge AI:** Runs on local devices; Tiny AI = runs on microcontrollers
- **LLMs:** Use transformers, attention, next-token prediction
- **Prompt Engineering:** Be specific, give context, use examples, assign roles, chain of thought
- **RAG:** Retrieve documents + Generate answers = accurate, up-to-date
- **Hallucination:** Confident false outputs; mitigate with RAG, fact-checking, human review
- **XAI:** Feature importance, LIME, SHAP, attention maps, counterfactuals
- **Bias:** Data bias, algorithmic bias, interaction bias; must be actively mitigated
- **Responsible AI:** Fairness, Privacy, Security, Governance, Ethics
- **Ecosystem:** Python → TensorFlow/PyTorch → Hugging Face → Ollama/LangChain → Vector DBs
- **Future Trends:** AGI, Sustainable AI, Neuro-Symbolic AI, Regulation, Quantum ML

Quick Formula Reference

Bayes' Theorem:

$$P(A|B) = P(B|A) * P(A) / P(B)$$

A* Search:

$$f(n) = g(n) + h(n)$$

Time Complexities:

- BFS: $O(b^d)$
- DFS: $O(b^m)$
- A*: $O(b^d)$ [with good heuristic, explores far fewer nodes]

Space Complexities:

- BFS: $O(b^d)$
 - DFS: $O(bm)$
-

Exam Tips for 25-Mark CA

1. **Read the scenario carefully.** The answer is always hidden in the details.
 2. **Identify the key concept being tested.** Is it agent types? Search algorithms? Knowledge representation?
 3. **Eliminate obviously wrong options first.** This increases your chances dramatically.
 4. **Watch for “always,” “never,” “only.”** These are often traps.
 5. **Remember: All current AI is Narrow AI (ANI).** Any option mentioning AGI or ASI as existing is WRONG.
 6. **BFS = queue, level-by-level; DFS = stack, deep-first.** This distinction is tested frequently.
 7. **A* uses BOTH $g(n)$ and $h(n)$.** Best First uses ONLY $h(n)$.
 8. **Forward chaining = data-driven; Backward chaining = goal-driven.**
 9. **Bayes’ Theorem questions often test the base rate fallacy.** Rare disease + positive test high probability.
 10. **RAG prevents hallucination by grounding answers in real documents.**
 11. **XAI is about explainability, not accuracy.**
 12. **Edge AI = local processing; Cloud AI = remote processing.**
 13. **When in doubt, think about what a rational agent would do with the given information.**
-

Good luck with your CA! You have got this!

This book was written to help every LPU student understand AI without needing a teacher. If you read every unit, solved every MCQ, and understood the explanations, you are ready to ace your exam.

Remember: AI is not magic. It is math, logic, and data working together. Once you understand the basics, everything else falls into place.

Happy Learning!